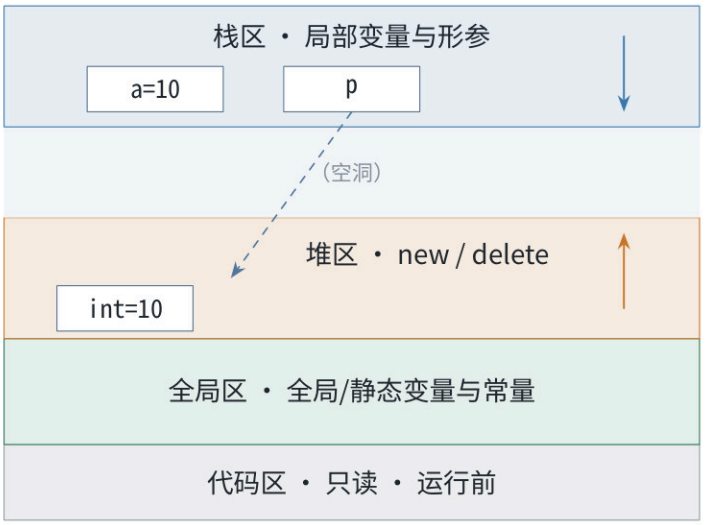
C++ 初步与进阶

内存分区模型 · 引用 · 函数提高

可视化零基础讲义



课程《高级程序设计与实践》实验一：C++初步知识（41 页）

全书一句话： C++ 让你“更灵活地编程”，代价是你得知道**数据放在哪里、活多**

阅读指南

定义　怎样使用这本讲义

- 先读每章开头的**来源**：它告诉你这一节对应你笔记的哪一页（Memo No.）或课件的哪一页。
- 每个重要知识点后都有：一张图、一个“生活中的它”、一个“对照”（和相近概念比较，含 ✔ △ ✘ 迁移判断）。
- 做题前先读“学完这里，你应该能够回答”；做完题再看答案与图解。

盒子与颜色的约定（颜色和边框就是身份牌，看一眼就知道这块内容的“地位”）：

外观	含义
定义 核心公式	深蓝细框：定义、公式、结论（最重要）
直观理解	左侧蓝条：直观理解、为什么需要、教学补全、知识联系、对照
生活中的它	暖米白：生活类比、例题、学科小史、从课本到现实
易错点 / 原图纠错	极浅红：易错点；对原笔记或课件的订正（会写明你原来写的、实测结果）
小结 / 自检	极浅绿：小结、学完能回答的问题

✔ 成立　△ 有条件成立　✘ 不成立

教学补全　讲义里的“新内容”从哪来

- 标注**教学补全**的是我补充的，不在你的笔记或课件里；
- 标注**原图纠错**的是你的笔记或课件里原有的笔误、过时写法或不精确之处，并给出订正；
- 带“实测”的结论，都是我在 g++ 13（`-std=c++17`）下实际编译运行得到的；地址、耗时会随机器和每次运行变化，但规律不变。

目录

阅读指南 1

学习路线与前置知识 4

课程背景（PDF 第 1–4 页） 4

第 1 章　程序骨架与输入输出 5
1.1　最简单的 C++ 程序 5
1.2　cin：输入语句 6
1.3　cout：输出语句与缓冲区 8
1.4　输出时的类型判断与默认格式 9
1.5　控制符：规定域宽、小数位、对齐 10
1.6　带类的 C++ 程序（封装初识） 11

第 2 章　const：只读的常变量 12
2.1　什么是常变量 12
2.2　const 与指针：两种“固定” 13
2.3　const 修饰的全局与局部常量，住在哪里 14

第 3 章　内存分区模型 14
3.1　四个区域总览 14
3.2　代码区：程序运行前 16
3.3　全局区：程序运行前 17
3.4　栈区：编译器自动管理 19
3.5　堆区：程序员自己管 20

第 4 章　引用：给变量起别名 22
4.1　引用的基本语法 22
4.2　引用的注意事项 23
4.3　引用做函数参数 25
4.4　引用做函数返回值 26
4.5　引用的本质 28
4.6　常量引用 29
4.7　PDF 补充：引用的更多规则 30

第 5 章　函数提高：默认参数、占位参数、重载与内联 31
5.1　函数默认参数 31
5.2　函数占位参数 33
5.3　函数重载 34
5.3.1　注意事项（笔记 494） 36
5.4　内联函数 37

第 6 章　字符串类型 string 39
6.1　为什么需要 string 39

6.2 定义、赋值与访问 . . . 39
6.3 输入输出与运算 . . . 40

第 7 章 综合：把内存、引用与函数串起来 . . . 41
7.1 一张概念地图 . . . 42
7.2 函数怎样把结果交出来：四种方式对照 . . . 42
7.3 综合示例：把本讲义用一遍 . . . 42
7.4 从课本到现实：为什么这些细节值钱 . . . 43

第 8 章 正式练习：38 题 . . . 44
A 基础巩固训练（25 题） . . . 44
B 中等偏下提升训练（10 题） . . . 50
C 跨学科迁移训练（3 题） . . . 53

第 9 章 答案速查与逐题图解解析 . . . 54
答案速查 . . . 55
解析 A 基础巩固（1–25） . . . 56
解析 B 中等偏下提升（26–35） . . . 63
解析 C 跨学科迁移（36–38） . . . 66

第 10 章 覆盖检查表 . . . 67
知识点 × 题目覆盖表 . . . 67
原稿覆盖对照 . . . 68
可视化覆盖检查 . . . 68

学习路线与前置知识

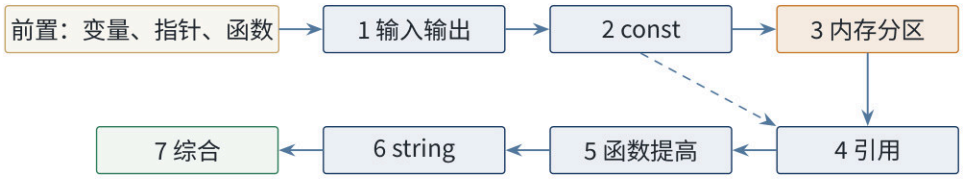


图 1　学习路线：先打地基（1–2），再理解内存（3），然后引用、函数、字符串（4–6），最后综合（7）

定义　前置知识 5 分钟回顾

- **变量**：给一块内存起名字。`int a=10;` 创建一块能放整数的内存并起名 a。
- **地址**：每块内存的编号，`&a` 取得它。
- **指针**：专门存放地址的变量，`int *p=&a;`，用 `*p` 访问所指的值。
- **函数**：一段有名字的代码，参数是传进去的数据，返回值是传回来的结果。
- **作用域**：一个名字在哪段代码里有效，大括号 {} 圈出一个作用域。



图 2　前置回顾：变量 a、地址与指针 p（地址为示意）

课程背景（PDF 第 1–4 页）

课件首页是《高级程序设计与实践》 —— 面向对象编程概述（相关学院　　）。第 2 页列出了两本参考书：**《C++面向对象程序设计》（第 4 版，谭浩强编著，清华大学出版社）**与**《C++面向对象程序设计实验和课程设计教程》**。

一、基础性实验	实验一 简单程序开发；二 类与对象；三 继承与派生；四 函数重载与运算符重载；五 虚函数与多态性；六 C++流和文件流；七 类型转换和模版；八 异常处理和命名空间
二、设计性案例实验	21 点纸牌游戏；电话号码簿；石头-剪刀-布游戏；骰子游戏；万年历
三、学以致用大赛	3 人一组，合作开发，提交作品
注意事项	上课必须带电脑

成绩构成（PDF 第 4 页）： 理论基础课 8 个基础实验 ×4 分（32%）；5 个综合实验 ×6 分（30%）；综合实验答辩 8 分（8%，不报名展示者该环节得基础分 4 分）；学以致用大赛 30 分（30%）。实验报告为 pdf 版，包含源代码及运行情况，按模板编写；延后补交每份最多得 2 分（综合实验最多 3 分）；**实验报告雷同 90% 以上，该次实验为 0 分**。最终成绩百分制，除综合实验答辩外，其他三项缺少任何一个环节即为不及格。 **本实验（实验一：C++初步知识，PDF 第 5–6 页）的目的：** 1 掌握 C++ 的输入输出；2 了解 const 定义常变量；3 掌握函数的重载、内置函数、函数的默认值参数；4 掌握变量的引用；5 了解字符串类型；6 了解动态分配撤销内存。——本讲义第1–6章与之一一对应，第3章（内存分区）来自你的笔记。

学科小史　C++ 是怎么长成今天这样的



图 3　C++ 简史（据 Wikipedia 与 cplusplus.com 的 C++ 历史页）

“带类的 C”这个名字很说明问题：C++ 从一开始就是在 C 的基础上，加上“类、内联、默认参数”等让大程序更好管理的工具。这也是本讲义的几个主角（inline、默认参数、重载、引用、new/delete）出身相同的原因：它们都是为了**少写出错的代码、更直接地表达意图**。后来标准库把 string 带进来，让你不必再手工管理字符数组。

第 1 章　程序骨架与输入输出

来源 课程 PDF 第 6–18 页（实验目的、最简程序、cin / cout、控制符）。 本章是后面所有章节的“地基”：你先要能读懂一个 C++ 程序长什么样，并且知道数据怎样“进来”、怎样“出去”。

教学补全　本章在整份讲义里的位置

课程 PDF 的“实验目的”列了 6 件事：**输入输出、const、函数的重载/内联/默认参数、引用、字符串、动态分配内存**。 你的手写笔记则按另一条线索展开：**内存分区模型→引用→函数提高**。 本讲义把两条线合并：先打地基（第 1–2 章），再讲内存（第 3 章），然后是引用、函数提高、字符串（第 4–6 章），最后第 7 章把它们串起来。

1.1　最简单的 C++ 程序

定义　程序骨架

一个最小的 C++ 程序包含：#include <iostream>（引入输入输出库）、using namespace std;（声明使用标准命名空间）、int main()（程序入口）以及 return 0;（正常结束返回 0）。

```
#include <iostream>          // 头文件：标准库不带 .h
using namespace std;         // 、coutcin 等都定义在命名空间 std 里
int main()                   // 函数类型 ： 返回给操作系统的状态码int
{
    int a, b;
    cin >> a >> b;           // 输入：例如 12 -3
    int sum = a + b;
    cout << "a+b=" << sum << endl;   // 输出 a+b=9
    return 0;                // 返回 0 表示正常结束
}
```

PDF 第 7 页 的运行示例是输入 12 -3。注意：该页示例输出写成 a+b=9，而 12+(-3)=9，是一致的。

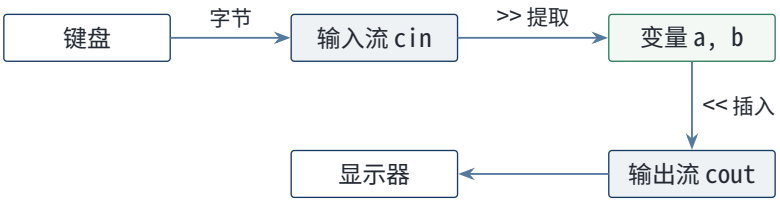


图 4　数据流向：键盘 → cin → 变量 → cout → 屏幕

直观理解

把 cin 想成一根从键盘通向程序的“水管”，>> 是水龙头：每拧一次，从水管里取出一段数据放进变量。 cout 是另一根通向屏幕的水管，<< 是把东西倒进去。箭头 >> 与 << 的“开口方向”恰好就是数据流动的方向。

生活中的它

超市收银：顾客把商品放上传送带（cin），收银员逐件扫码取走（>>），小票从打印机里一条条吐出来（cout、<<）。商品之间只要隔开就行，空格、回车都能充当“分隔”。

对照：iostream 与 iostream.h、int main 与 void main

	<iostream>	<iostream.h>
属于	标准 C++	早期（VC6 时代）写法
命名空间	需要 using namespace std	无需
现代编译器	✔ 通过	✘ 多数不再支持

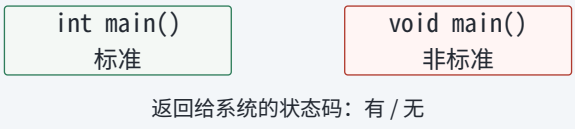


图 5　对照：标准写法与旧写法

迁移判断：

规则	判断
“头文件不带 .h”适用于 C++ 标准库头文件	✔
同样适用于 C 的头文件（旧写法 <stdio.h>）	△ 可用，但推荐 <cstdio>
“main 可以不返回 int”	✘ 标准要求 int

原图纠错　PDF 第 7、11、15 页

课件里出现 #include <iostream.h> 与 void main()。我在 g++ 13（-std=c++17）下验证：iostream.h 已不被支持，void main 虽然部分编译器（如旧 VC）放行，但不是标准写法。 本讲义所有示例统一用 <iostream> 与 int main()。

1.2　cin：输入语句

PDF 第 9–11 页

定义　cin 语句

`cin >> 变量1 >> 变量2 >> ...>> 变量n;` 由输入流对象 `cin` 和**提取运算符** `>>` 实现输入。一个 cin 语句可以分写成若干行。输入时用**空格、回车、Tab**分隔数据；系统根据变量的类型，从输入流中提取相应长度的字节。

PDF 第 10 页的例子：

```
char c1, c2;  int a;  float b;
cin >> c1 >> c2 >> a >> b;
cout << c1 << '\t' << c2 << '\t' << a << '\t' << b << endl;
```

- 输入 `1 2 34 56.78`：`c1='1'`（内存里是 ASCII 码 49），`c2='2'`（50），`a=34`，`b=56.78`；
- 输入 `1234 56.78`：`c1='1'`，`c2='2'`，`a=34`，`b=56.78`——因为 char 一次只取 1 个字符，a 取剩下的 34。



图 6　输入 `1 2 34 56.78` 时输入流被依次“吃掉”（空格被跳过）

易错点

`cin` 读 char 时**会跳过空格、回车、制表符**。如果你就是想读入一个空格字符，要用 `c1 = getchar();` 或 `cin.get(c1);`（PDF 第 10–11 页）。

对照：`cin >> c` 与 `cin.get(c)` / `getchar()`

输入 "1 2 34 56.78"	c1,c2 用 cin>>	用 get/getchar
c1	'1'	'1'
c2	'2'（空格被跳过）	' '（空格本身被读入）
a	34	2（读到 2，后面是空格）

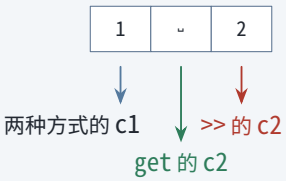


图 7　同一份输入，两种读法的落点不同（课件 PDF 第 11 页输出里的“␣”就是这个空格）

迁移判断： `cin >> int` 也跳过前导空白 ✔；`cin >> string` 也跳过前导空白、遇空白停止 ✔（见第 6 章）；`cin.get()` 逐字节读取不跳过 → 与上表一致 ✔。

生活中的它

就像在电影院排队检票：`cin >>` 的检票员会自动忽略队伍里的空位（空白），而 `cin.get` 的检票员连空位也要登记一下。

1.3　cout：输出语句与缓冲区

PDF 第 12–15 页

定义　cout 语句

`cout << 表达式1 << 表达式2 << …;`，由输出流对象 `cout` 和**插入运算符** `<<` 实现输出；同样可以分写成若干行。

直观理解　缓冲区：先攒着，一起出

`cout` 先把要输出的数据**顺序**放进内存中的**输出缓冲区**，并不立即显示；直到缓冲区满，或遇到 `endl`、`cout.flush()`，才把里面的数据一起送到显示器并清空缓冲区。

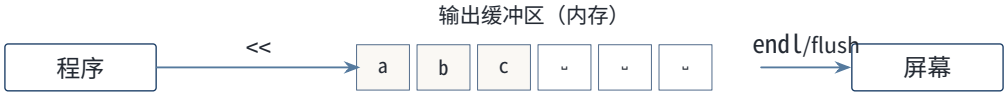


图 8　数据先进缓冲区；遇到 `endl` 才一起出去

生活中的它

外卖骑手不会每做好一单就出发，而是等一批单子攒够了（或店员喊“出餐！”）才一起送——`endl` 就是那声“出餐”。

对照：`'\n'` 与 `endl`

	`'\n'`	`endl`
换行	✔	✔
立即刷新缓冲区	✘	✔
典型用途	大量输出时更快	需要立刻看到结果（如调试）

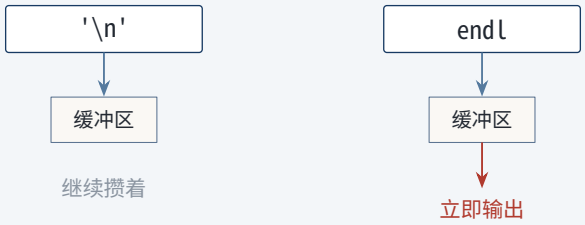


图 9　两种换行：是否强制“出餐”

迁移判断： `cout.flush()` 与 `endl` 的“刷新”部分相同 ✔；`cerr` 默认不缓冲 △（课件未涉及，属教学补全）。

原图纠错　PDF 第 13 页：“cin»a, b; 编译能通过，但运行错误！b 得不到值”

我在 g++ 下实测：`cin >> a, b;` 被解析成**逗号表达式** `(cin >> a), b`。编译只给出 `-Wunused-value` 警告，运行时**没有崩溃**，只是 `b` 根本没有被读入（保持原值）。所以更准确的说法是：“语法被当作逗号表达式，`b` 得不到值，是**逻辑错误**而非运行时错误”。同理 `cout << a, b, c;` 只输出 `a`。

1.4　输出时的类型判断与默认格式

PDF 第 14–15 页

定义　默认输出规则

- 整型数据：默认按**十进制**输出；
- 字符型数据：按字符形式输出；
- 浮点型：先化成规范化指数形式，若**指数 > -5**，按十进制小数形式输出；若**指数 ≤ -5**，按规范化指数形式输出；默认最多 **6 位有效数字**。

PDF 第 15 页的程序及我实测的输出（g++ 13）：

```
int a = 0x12;                 // 十六进制字面量 = 十进制 18
float b = 16.912375678;
double c = 34.5678e-5;        // = 3.45678e，指数-4 -4
double d = 9.765487e-5;       // 指数 -5
int e;                        // 未赋初值：值不可预测
```

变量	输出（实测）
a	a=18
b	b=16.9124（6 位有效数字）
c	c=0.000345678（指数 $-4 > -5$，小数形式）
d	d=9.76549e-05（指数 $-5 \le -5$，指数形式）
e	不确定的垃圾值

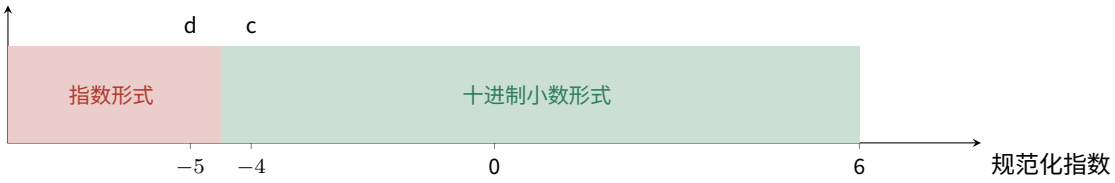


图 10　输出形式由指数位置决定（-5 是分界；上界由有效位数 6 决定，属教学补全）

教学补全　教学补全：规则从哪来

课件说“指数 > -5”，等价于标准库规则中的 %g 风格：指数 < -4 用科学计数法，指数 $\ge$ 精度（默认 6）也用科学计数法。所以 1234567.0 会输出 1.23457e+06。

易错点　未初始化的变量

int e; 里面不是 0，而是内存里原来残留的数据。读取它是**未定义行为**，永远先赋初值。

1.5　控制符：规定域宽、小数位、对齐

PDF 第 16–18 页 使用控制符要额外 `#include <iomanip>`。

控制符	作用
`dec / hex / oct`	整数基数设为 10 / 16 / 8
`setiosflags(ios::scientific)`	浮点数以指数形式显示
`setiosflags(ios::fixed)`	浮点数以小数形式显示，并固定小数位数
`setprecision(n)`	默认形式：n 为有效数字；fixed / scientific：n 为小数点后位数
`setw(n)`	字段宽度（域宽）为 n 位
`setiosflags(ios::left / right)`	输出左对齐 / 右对齐
`setiosflags(ios::uppercase / lowercase)`	指数 E、十六进制字母用大写 / 小写
`resetiosflags(X)`	取消控制符 X

结论　有效期

只有 `setw` 仅对其后的 1 个输出项有效；其余控制符对后续所有输出项都有效，直到被改成其他设置。

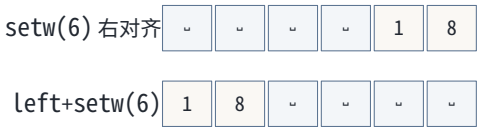


图 11　域宽 6 位：浅灰为补的空格；右对齐空格在左，左对齐空格在右（实测输出 " 18|18 |"）

```
#include <iomanip>
cout << hex << 18 << ' ' << oct << 18 << ' ' << dec << 18;  // 12 22 18
cout << setw(6) << 18 << '|';                               //      18|
cout << setiosflags(ios::fixed) << setprecision(2) << 3.14159; // 3.14
```

对照：`setprecision` 在两种模式下的含义

数值 3.14159	默认 setprecision(2)	fixed+(2)	scientific+(2)
输出	3.1	3.14	3.14e+00
n 的含义	有效数字	小数位	小数位



图 12　同一个 n=2，在不同模式下“数”的起点不同

迁移判断： `setw` 同样只管一项 ✔；`setprecision` 一旦设置持续生效 ✔；`fixed` 与 `scientific` 可以叠加 ✘ ——见下。

原图纠错　叠加 fixed 与 scientific 的陷阱——教学补全

我在 g++ 下依次设置 ios::fixed 再设置 ios::scientific，没有用 resetiosflags 取消前者，实测输出变成了 0x1.34ap+10（十六进制浮点格式）。两者同时置位在标准里代表 hexfloat。想换格式必须先 resetiosflags(ios::floatfield)。

1.6　带类的 C++ 程序（封装初识）

PDF 第 8 页

```
class Student {                 // 声明一个类 ，实现了封装Student
private:                        // 私有成员：信息数据()隐蔽
    int num;                    // 学号
    int score;                  // 成绩
public:                         // 公有成员
    void setdata() { cin >> num; cin >> score; }          // 输入数据
    void display() { cout<<"num="<<num<<endl; cout<<"score="<<score<<endl; }
};                              // 类声明以分号结束!
Student stud1, stud2;           // 全局对象：定义类的全局变量
int main() {
    stud1.setdata(); stud2.setdata();   // 输入 1001 98 与 1002 76
    stud1.display(); stud2.display();
    return 0;
}
```

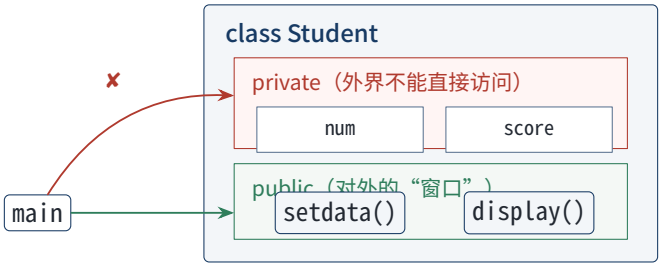


图 13　封装：数据在私有区，只能通过公有函数访问

生活中的它

银行柜台：账户余额锁在后台保险柜（private），柜台玻璃上只留存款、取款的小窗口（public 函数）。你不能直接伸手去摸钱，只能“申请”。

对照：类的 private 与 public（对照结构体 struct）

	class 默认	struct 默认
成员访问权限	private	public
可以加成员函数	✔	✔（C++ 中）
可以用 private: 修改默认	✔	✔



图 14　两者只在“默认权限”上不同（教学补全；课件只讲了 class）

迁移判断： 在 C 语言中 struct 不能含函数 ✘；封装的思想适用于两者 ✔；“未写访问符就一律私有”只对 class 成立 △。

学完这里，你应该能够回答

1. cin >> 与 cout << 的“开口方向”各表示什么？
2. 为什么输入 1 2 34 56.78 时 c2 读到的是 '2' 而不是空格？
3. '\n' 与 endl 差在哪？setw 与 setprecision 的有效期有何不同？
4. 类里的 private 成员为什么不能被 main 直接访问？

第 2 章　const：只读的常变量

来源 PDF 第 19 页；笔记 Memo 481–482（const 修饰的全局/局部常量）、Memo 489（指针常量回顾）。

2.1　什么是常变量

定义　常变量（只读变量）

定义变量时加上关键字 const，该变量的值在程序运行期间**不能改变**，称为**常变量**（constant variable），又称**只读变量**。定义时**必须同时初始化**，此后值不能再改变。 const int a=3; 与 int const a=3; 写法等价（PDF 第 19 页）。

```
const int a;          // ✘ 定义时必须初始化
int b = 5;
const int a = 2 + b;  // ✔ 初始值可以是表达式的值（运行时才算也可以）
a = 20;               // ✘ 不能企图修改常变量的值
b = a * 10;           // ✔ 读 a 当然可以
```

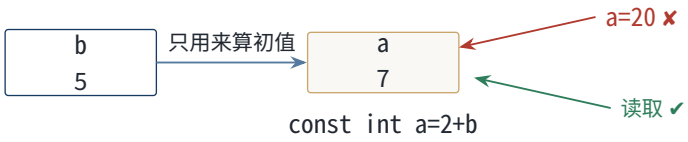


图 15　常变量：初始化后“上锁”，只能读不能写

为什么需要它

有些值本来就不该变：圆周率、一周有 7 天、数组长度。把它们写成 const，一旦你（或同事）手滑写出 PI = 3;，编译器立即报错，而不是等程序算错了才发现。

生活中的它

身份证号：出生时就定下，不能改。快递柜的取件码也是“只读”——你只能拿它开柜，不能自己改一个。

对照：常变量 const int a 与宏 #define A 3

	const int a=3;	#define A 3
有类型检查	✔	✘
有自己的内存空间	△ 常需要（可被优化掉）	✘ 纯文本替换
受作用域限制	✔	✘ 从定义处到文件尾
调试器能看到名字	✔	✘



图 16　两种“常量”：一个是变量的特殊形态，一个只是文字替换

迁移判断： 宏同样可以让代码“用名字代替魔法数字” ✔；宏可以限定类型 ✘；const 可以放进名字空间 / 类里 ✔（课件未涉及，教学补全）。

2.2 const 与指针：两种“固定”

笔记 Memo 489 的“回顾”部分，PDF 第 19 页底部也给出了一篇相关博客（const int *p 与 int *const p 的区别）。

定义　常量指针 与 指针常量

- const int *p（常量指针）：指向的值不可改，指针可以改指向。
- int *const p（指针常量）：指针本身不可改指向，指向的值可改。

记忆法：看 * 的位置。const 在 * 左边，管“东西”；在 * 右边，管“指针”。

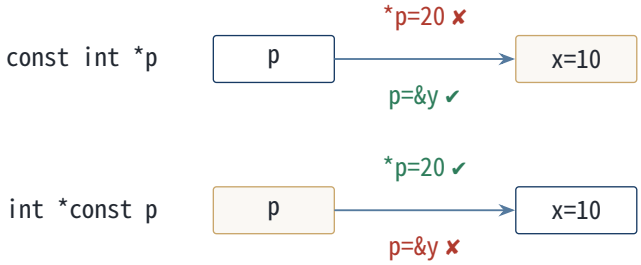


图 17　浅黄色标出“被 const 锁住”的部分：常量指针锁住数据，指针常量锁住指针

生活中的它

常量指针像“博物馆的展品导览牌”：牌子可以换指向另一件展品，但你不能去动展品本身。指针常量像“焊死在桌上的台灯”：灯泡（值）可以换，台灯位置不能挪。

对照：常量指针、指针常量、两个都 const

	const int *p	int *const p	const int *const p
能改 *p	✘	✔	✘
能改 p 的指向	✔	✘	✘
定义时必须初始化	✘	✔	✔

迁移判断： 引用的本质是 int *const（见第 4 章）✔；“引用必须初始化、不可更改指向”恰与

> “指针常量必须初始化”呼应 ✔；引用能被重新绑定 ✘。

易错点　术语陷阱

“常量指针”与“指针常量”的中文叫法在不同教材里偶尔相反。本讲义与你的笔记一致：`const int *p` 称**常量指针**，`int *const p` 称**指针常量**。遇到争议时，直接写出声明比说名字更可靠。

2.3　const 修饰的全局与局部常量，住在哪里

笔记 Memo 481–482 这里先给结论，第3章会用图解释：

变量种类	位置	说明
全局变量、静态变量	全局区	笔记 482 的“分类①”
const 修饰的全局变量	全局区（只读常量区）	同上
局部变量	栈区	笔记 482 的“分类②”
const 修饰的局部常量	栈区（不在全局区）	同上

学完这里，你应该能够回答

1. 为什么 `const int a;` 不能通过编译？
2. 初始值可以是表达式吗？例：`const int a = 2+b;`
3. `const int *p` 与 `int *const p` 谁锁数据，谁锁指针？

第3章　内存分区模型

来源 笔记 Memo 481–485（第一章：内存分区模型）；PDF 第 39–40 页（new/delete）。 笔记的意义一栏写得很好：**更大灵活的编程**——理解四个区域，才能决定“数据放哪里、活多久”。

3.1　四个区域总览

定义　C++ 程序运行时的四个区域（笔记 481）

1. **代码区**：存放函数体的二进制代码（程序运行前就有）。
2. **全局区**：存放全局变量、静态变量以及常量（程序运行前就有）。
3. **栈区**：由编译器自动分配和释放，存放函数的参数值、局部变量等。
4. **堆区**：由程序员分配与释放；若不释放，将在程序结束时由操作系统回收。

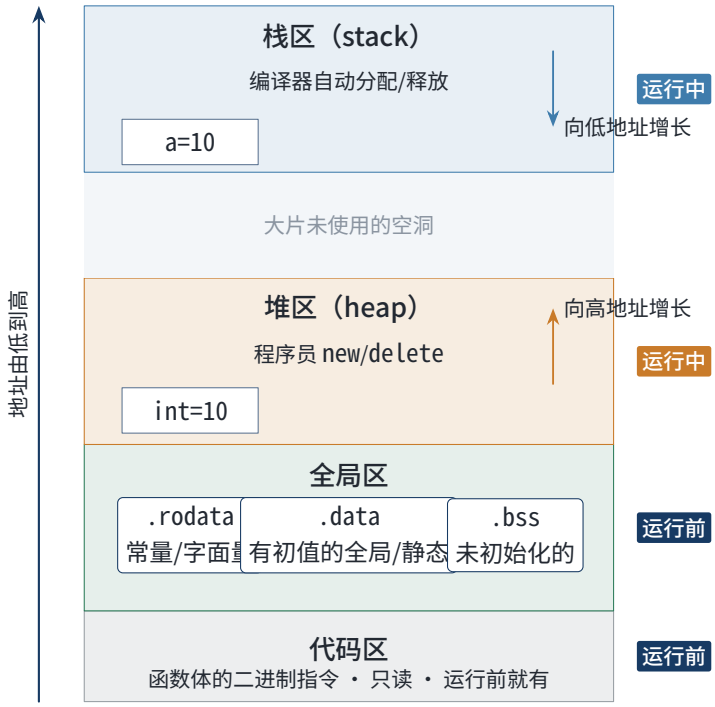


图 18　进程地址空间示意（Linux 上典型布局；各区之间的间隔不按比例）

教学补全　教学补全：为什么栈在上、堆在下？

Linux 典型布局中，代码和全局数据地址最低，堆紧接其后向高地址长，栈在最高处向低地址长。两者相向而行，中间是空洞，谁也不容易撞到谁。笔记 481 把四个区画成横向 4 个格子（1–4），这是“示意”，真实内存是一条线性的地址空间。

直观理解

把程序想成一栋写字楼：**代码区**是总经理办公室的“规章制度手册”（只读，大家共用一本）；**全局区**是公共储物间，开业前就备好、打烊后才收拾；**栈区**是前台的“便签板”，随用随贴、用完即撕，由前台（编译器）自动打理；**堆区**是租来的仓库，由你自己（程序员）决定租多大、什么时候退租。

生活中的它

酒店：公共区域（全局区）开业前就布置好；客房（栈）住客退房自动清洁；长租仓库（堆）要你自己归还钥匙，不还就一直占着，最后由酒店（操作系统）打烊时统一回收。

对照：栈区 与 堆区

	栈区	堆区
谁分配/释放	编译器自动	程序员（new/delete）
存放内容	参数值、局部变量	new 出来的数据
生命周期	函数结束即释放	直到 delete；不释放则程序结束时由系统回收
典型错误	返回局部变量地址	忘记 delete（泄漏）
大小	较小（本机 8 MB 上限）	很大（受物理/虚拟内存限制）



图 19　对照：谁来“收拾房间”

迁移判断： “自动变量用完即释放”对局部变量成立 ✔，对 static 局部变量不成立 ✘（它住全局区）；“堆里的数据会被 C++ 自动回收” ✘ 对裸 new 成立吗？不成立，只在程序结束时由操作系统回收 △。

3.2　代码区：程序运行前

笔记 Memo 481

定义　代码区

1. 存放 CPU 执行的**机器指令**；
2. 代码区是**共享**的：对于频繁被执行的程序，内存中只需有一份代码；
3. 代码区是**只读**的，原因：防止程序意外地修改它的指令。

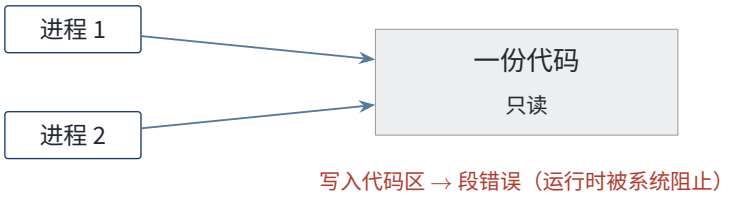


图 20　代码区：多个进程可共享同一份只读代码

生活中的它

学校图书馆里的《新华字典》：一本放在阅览室，所有人都来查（共享），但不允许涂改（只读），否则后来的人就查错了。

对照：代码区 与 全局区常量区

	代码区	常量区
存什么	指令	字符串字面量、const 全局
只读	✔	✔
可执行	✔	✘
共享	✔	△



图 21　两块“只读”区域，但内容与权限略有不同

迁移判断： “都在程序运行前就存在” ✔；“都不能被程序修改” ✔；“里面都是函数体” ✘ 只有代码区是。

3.3　全局区：程序运行前

笔记 Memo 481–482

定义　全局区

1. 全局区存放**全局变量**和**静态变量**；
2. 全局区还包含**常量区**，字符串常量和其他常量也存放在此；
3. 该区域的数据在程序结束后由操作系统释放。

笔记 482 的示例（原样抄录；订正见下方的“原图纠错”）：

```
int global_var = 20;                // 全局变量
const int const_global_var = 30;    // const 修饰的全局常量
int main() {
    int a = 10;
    cout << "局部变量 a 的地址: " << (long long)&a << endl;
    cout << "全局变量 global_var 的地址: " << (long long)&global_var << endl;
    cout << "const_global_var 的地址: " << (long long)&const_global_var << endl;
    static int static_var = 17;                         // 静态变量
    cout << (long long)&static_var << endl;
    string str = "Hello, World!";                       // 字符串常量
    cout << (long long)&str << endl;
    const int c = 21;                                   // const 修饰的局部常量
    cout << (long long)&c << endl;
    return 0;
}
```

分类（笔记 482）： ① 在全局区：全局变量、静态变量、const 修饰的全局变量；② 不在全局区：局部变量、const 修饰的局部常量。

我把这份程序补全 `#include <string>` 并改正后在 g++ 13 下运行，得到下面的**真实地址**（十六进制，每次运行前缀会随 ASLR 改变，但“谁高谁低”的顺序不变）：

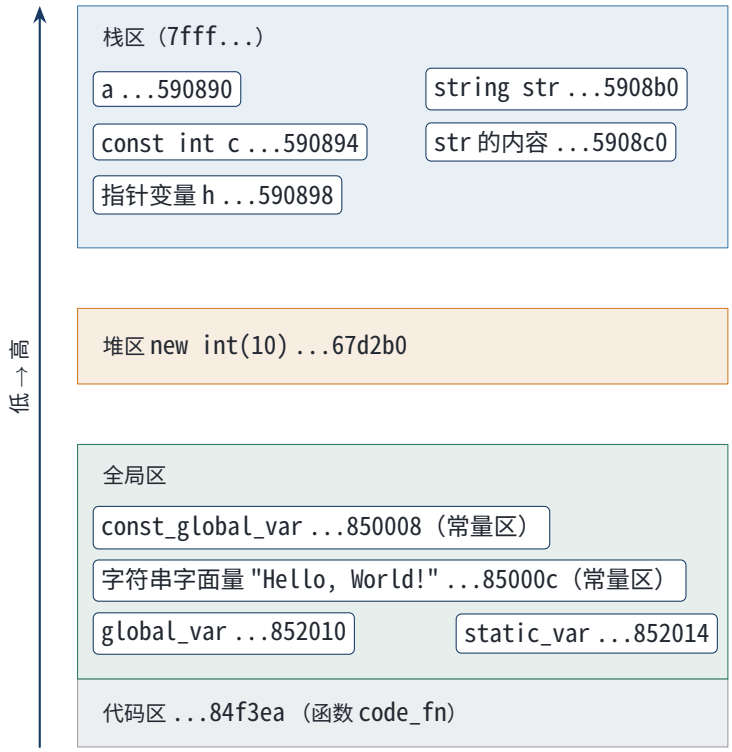


图 22　同一次运行实测的各变量地址（省略前缀）：代码 < 常量 < 全局/静态 < 堆 < 栈

原图纠错　笔记 482：string str 的地址不是“字符串常量”的地址

笔记把 `String str = "Hello, World!"` 与 `&str` 放在“字符串常量”下，但这里有三点需要订正：

1. 类型名应写成小写 `string`（需要 `#include <string>`），`String` 在标准 C++ 里不存在；
2. `&str` 是string 对象本身的地址，它是局部变量，在栈区（实测 `...5908b0`）；
3. 真正的字符串常量 `"Hello, World!"` 在常量区（实测 `...85000c`）。要看到它的地址应当用 `const char *s="Hello, World!"; cout<<(long long)s;`。

另外这个 13 个字符的字符串因为短，被 libstdc++ 的“短字符串优化（SSO）”直接存放在 `string` 对象内部（实测 `str.data()` 比 `&str` 恰好大 16 字节），所以它的内容也在栈上。

原图纠错　笔记 482：局部 const 常量

笔记正确地指出了`const` 修饰的局部常量不在全局区；实测 `c` 地址与 `a` 相邻（栈上）。

生活中的它

商场“寄存柜”分两种：一种是整栋楼固定的消防栓、灭火器位置（全局区，开业前就有），另一种是顾客自己带来的购物袋（栈上的局部变量），顾客一走就清空。

对照：全局变量、静态变量、局部变量

	全局变量	静态变量 static	局部变量
所在区域	全局区	全局区	栈区
生命周期	整个程序	整个程序	所在函数/块
可见范围	整个程序（有声明）	所在函数/文件	所在块
未初始化时	自动清零	自动清零	垃圾值



图 23　寿命相同的两个“常住”居民和一个“临时”住户

迁移判断： static 局部变量只初始化一次 ✔；static 局部变量能在函数外直接访问 ✘（只是寿命长，可见范围仍是函数内）；全局 const 与全局普通变量同在全局区 ✔（常量区 vs 数据区，且常量区只读）。

3.4　栈区：编译器自动管理

笔记 Memo 482–483

定义　栈区

特点：由编译器**自动分配释放**，存放函数的参数值、局部变量值等。 注意：**不要返回局部变量的地址**，栈区开辟的数据由编译器自动释放。

笔记 483 的示例：

```
int* func(int b) {          // 形参数据也会存放在栈区
    b = 100;
    int a = 10;             // 局部变量，存放在栈区，数据在执行完后自动释放
    return &a;              // a 的生命终结
}
int main() {
    int* p = func();        // 笔记写成 func()，缺少实参，应为 func(0)
    cout << "返回的地址: " << (long long)p << endl;
    return 0;
}
```

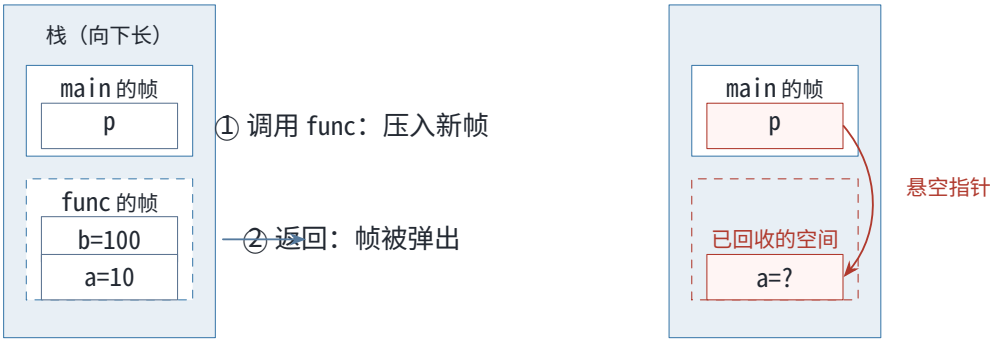


图 24　返回局部变量的地址：函数返回时帧被回收，p 指向了“已回收”的空间

生活中的它

退租的酒店房间：你手里还拿着门卡（指针），但房间已打扫、下一位住客可能入住。你用旧门卡进去，看到的可能是别人的东西——这就是悬空指针。

我的实测：把 func 的返回值写成 &a，g++ 编译会给出警告 address of local variable 'a' returned；运行时读 *p 的结果不可预测（可能仍是 10，也可能是垃圾值，也可能崩溃）。

原图纠错　笔记 483 示例的两处订正

1. func 有一个形参 int b，而 main 里写成了 func()，应改为 func(0)；
2. 笔记在 484 页补写的红字：“指针本质也是局部变量，放在栈上，指针保存的数据放在堆区”。更准确的说法是：**指针变量本身**放在栈上，**指针所指向的数据**放在堆区（前提是用 new 得到）。

对照：返回局部变量的“值”、“地址”、“引用”

	返回 int 值	返回 int*（&a）	返回 int&（a）
安全	✔	✘	✘
原因	拷贝了值	指向已消亡的栈空间	别名对应已消亡的栈空间



图 25　三种返回方式的安全性（引用见第 4 章）

迁移判断：　“返回局部变量的值是安全的”✔；“返回 static 局部变量的地址/引用是安全的”✔（它在全局区，见 4.4）；“返回 new 出来的指针是安全的”✔（但调用者要负责 delete）。

3.5 堆区：程序员自己管

笔记 Memo 483–485；PDF 第 39–40 页

定义　堆区与 new / delete

1. 特点：由程序员**分配与释放**；若程序员不释放，程序结束时由操作系统回收。
2. C++ 中主要利用 new **运算符**在堆区开辟内存。
3. new 类型 在堆上开辟，返回**该类型的指针**；delete 指针 释放。

核心公式　PDF 第 39–40 页 + 笔记 484

```
new 类型              // 生成指定类型的动态空间
new 类型初值()        // 生成动态空间并赋初值
new 类型元素个数[]    // 生成动态数组
delete 指针变量       // 释放 new 生成的单个动态空间
delete [] 指针变量    // 释放 new 生成的动态数组空间
```

笔记 483–485 的堆区示例（及 PDF 第 40 页的四个例子）：

```
int* func() {
    int* p = new int(10);     // 利用 new 关键字，可将数据开辟到堆区
    return p;                 // 返回的是堆上数据的地址：函数结束后数据仍在
}
int main() {
    int* p = func();
    cout << "p 指向的地址为: " << (long long)p << endl;    // 笔记原写“的地址”p
    delete p;                 // 用完要释放!
    return 0;
```

```
}
// PDF 第 40 页
int   *p1 = new int;            // p1 指向新开辟的整型空间
float *p2 = new float(3.14159); // 开辟 float 并放入初值 3.14159
char  *p3 = new char[10];       // 开辟 10 个字符的数组，返回首元素地址
int (*p4)[4] = new int[5][4];   // 开辟 5*4 的二维整型数组，返回首行的行地址
delete p1; delete p2; delete [] p3; delete [] p4;
```

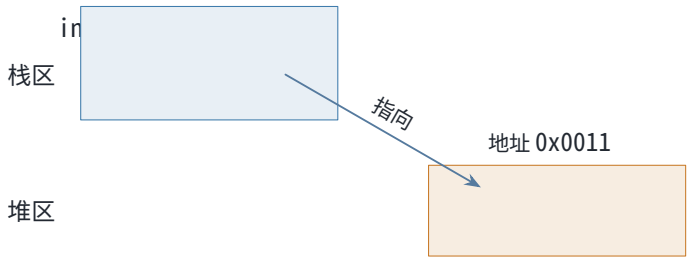


图 26　笔记 484 的图示：栈上的指针保存地址，堆上才是数据本身（地址为示意）

直观理解

栈上的 p 是一张“取件单”，上面写着快递柜编号（地址 0x0011）；堆里的 10 是柜子里的快递。丢了取件单（p 出了作用域也没 delete），快递就一直占着格子——这就是**内存泄漏**。

生活中的它

自己动手租仓库：new 是签租约拿到钥匙（指针），delete 是退租还钥匙。只丢钥匙不退租，仓库永远空占着钱（内存）。

原图纠错　笔记 484–485 的 Array_1 示例

笔记写了：

```
int* Array_1() {
    int* p = new int[10];
    for (int i = 0; i < 10; i++) p[i] = i;
    delete[] p;          // ✘ 先释放了
    return p;            // ✘ 又返回已释放的地址：悬空指针
}
```

delete[] p 之后再 return p，调用者拿到的是**已释放**的堆地址，使用它同样是未定义行为。应当**不在函数内 delete**，把释放责任交给调用者；或者直接返回 vector/智能指针（课外补充）。 另外 main 里 cout << "a地址: " << p1 输出的其实是 p1 所指的**堆上副本**的地址，不是变量 a 的地址；p1 与 p2 最后都没有 delete，会泄漏。

我用 AddressSanitizer / LeakSanitizer 实测过泄漏：忘记 delete 的程序在退出时被报告 “detected memory leaks”，而每次 new 得到的堆地址彼此不同——说明堆空间确实一直被占着。

原图纠错　PDF 第 39 页：“内存不足返回 NULL”

课件说 new 在内存不足时“返回一个空指针 NULL”。在标准 C++（C++98 起）中，new 失败默认**抛出 std::bad_alloc 异常**，只有 new (std::nothrow) T 才返回 nullptr。早期的 VC6 才返回 NULL。我在 g++ 下实测 new int[4e18] 得到 std::bad_alloc。 课件又说“new/delete 是运算符，不是函数，因此执行效率高”：new 内部会调用 operator new 函数分配原始内存，再执行构造。“运算

符”并不意味着自动更快；它的好处是会调用构造/析构函数（课程后续讲类时会用到）。

对照：delete p 与 delete[] p；malloc/free 与 new/delete

	new/delete	malloc/free
是运算符	✔	✘（C 库函数）
自动算大小	✔	✘
调用构造/析构	✔	✘
失败时	抛 bad_alloc	返回 NULL

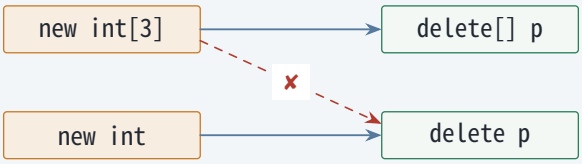


图 27　成对使用：配对错了是未定义行为

迁移判断： “每个 new 对应一个 delete” ✔；“new[] 必须用 delete[]” ✔；“malloc 出来的内存能用 delete 释放” ✘；“free(nullptr) 与 delete nullptr 都安全” ✔（教学补全）。

小结

变量种类	例	区域	谁释放	何时
全局/静态变量	int g; static int s;	全局区	系统	程序结束
常量	字面量、const 全局	常量区	系统	程序结束
局部变量/形参	int a;	栈区	编译器	函数返回
new 的数据	new int(10)	堆区	程序员	delete

学完这里，你应该能够回答

1. 四个区各存什么？哪两个在程序运行前就存在？
2. 代码区为什么要“共享”和“只读”？
3. 为什么返回局部变量地址危险？new 出的数据为什么没有这个问题？
4. “指针在栈上，指针指向的数据在堆上”怎么画？

第 4 章　引用：给变量起别名

来源 笔记 Memo 485–490（第二章：引用）；PDF 第 28–34 页（引用 Reference）。

4.1　引用的基本语法

笔记 485；PDF 28

定义　引用

引用就是给变量起一个**别名**。语法：数据类型 &别名 = 原名；例：int a=5; int &b=a; 此后 b 与 a 是同一个变量的两个名字：b=10 等价于 a=10；它们**共享同一块存储空间，具有同一地址**，即 &b 等于 &a。

```
int a = 10;
int &b = a;                        // b 是 a 的别名（引用）
cout << "a = " << a << endl;       // 10
cout << "b = " << b << endl;       // 10
b = 20;                            // 等价于 a = 20
cout << a << " " << (&a == &b) << endl;   // 20 1
```

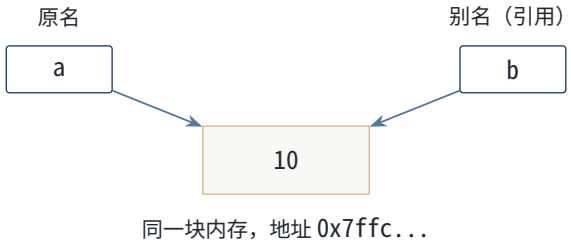


图 28　一个盒子，两张名牌：改任一名字下的值，另一个同时变化

生活中的它

你的本名叫“张三”，同学叫你“小张”，妈妈叫你“三三”。三个名字指向同一个人：有人叫“小张”去吃饭，来的当然是你。但世界上并没有多出一个人。

对照：引用 与 拷贝

	int &b=a;	int b=a;
是否新的存储空间	✘ 共用	✔ 新的一块
改 b 会影响 a	✔	✘
&a==&b	✔	✘



图 29　对照：别名与副本

迁移判断： 函数传参时，引用传参不拷贝 ✔，按值传参拷贝 ✔；“b=10 也会改 a”仅对引用成立，拷贝 ✘。

4.2　引用的注意事项

笔记 486；PDF 29–30

结论　两条铁律

1. 引用**必须初始化**（声明时就要说清它代表哪个变量）；
2. 引用**初始化后不可改变**——它不能再作为别的变量的引用。

```
int a = 10;
int &b;          // ✘ 必须要初始化
int &b = a;      // ✔
b = 30;          // 只是赋值，不是改绑定：a 变成 （笔记红字：只是赋值不是变化）30
int a1, a2;
int &c = a1;
int &c = a2;     // ✘ 企图使 c 又变成 a2 的引用（PDF 第 30 页，编译错误）
```

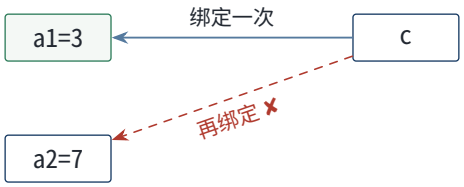


图 30　引用一旦绑定 a1，就不能再改去绑 a2

教学补全　PDF 第 29–30 页的补充条款

- 当引用作为**函数形参**时，不必在声明处初始化；它的初始化发生在**函数调用时**，形参引用成为实参的别名。
- 对引用的初始化，可以用一个变量，也可以用另一个引用：`int &b=a; int &c=b;`（c 是 b 的别名，也就是 a 的别名）。
- 区分“声明引用”还是“取地址”：`int &b=a;` 是引用（& 出现在**类型**位置）；`int *p=&a;` 是取地址（& 出现在**表达式**里）。

易错点　“赋值”与“绑定”不要混

`int &b=a;` 是**绑定**（只发生一次），之后每一次 `b = 值;` 都是**赋值**：改的是 a 里的值，b 仍然是 a 的别名。

生活中的它

给你取小名叫“小张”：一旦叫开了，不能第二天改成指向隔壁“小李”——那等于换人。你能做的是对“小张”这个人说“你把作业给我”（赋值）。

对照：引用 与 指针：能否改指向、能否为空

	引用	指针
必须初始化	✔（形参除外）	✘
可改变指向	✘	✔
可以为“空”	✘	✔（nullptr）
有独立的内存空间	✘（共享）	✔
访问方式	通过别名**直接**访问	通过地址**间接**访问

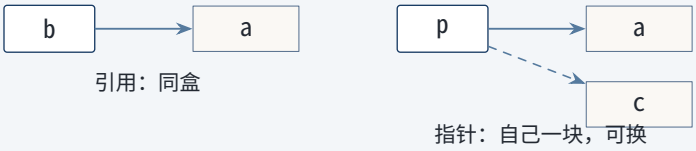


图 31　PDF 第 32 页表格的图示化

迁移判断： 引用“必须初始化”和“指针常量必须初始化”相同 ✔；“指针可以为空”对引用 ✘；“可以建立指针的引用”✔（见 4.7）；“可以建立引用的指针”✘ 同样是语法不允许（教学补全：不能有指向引用的指针，但可以取被引用变量的地址）。

4.3　引用做函数参数

笔记 486–488；PDF 第 31 页

定义　引用传参

作用：函数传参时，可以利用引用的技术让**形参修饰实参**（形参是实参的别名）。优点：可以**简化指针修改实参**的写法。

笔记 486–488 把“值传递、地址传递、引用传递”写在同一个 swap 例子里（这是我整理后的完整正确版；订正见下）：

```
// 值传递：形参是副本，换不了实参
void swap_01(int a, int b) { int temp = a; a = b; b = temp; }
// 地址传递：通过指针间接修改实参
void swap_02(int *a, int *b) { int temp = *a; *a = *b; *b = temp; }
// 引用传递（PDF 第 31 页）：形参 、ab 是实参的别名
void swap_03(int &a, int &b) { int temp = a; a = b; b = temp; }
int main() {
    int a = 10, b = 20;
    swap_01(a, b);   cout << "a=" << a << " b=" << b << endl;   // a=10 b=20 没变
    swap_02(&a, &b); cout << "a=" << a << " b=" << b << endl;   // a=20 b=10
    swap_03(a, b);   cout << "a=" << a << " b=" << b << endl;   // a=10 b=20
}
```

原图纠错　笔记 487：swap_02 的三处笔误

笔记写成：`int * temp = *a; *a = *b; *b = *temp;`。

1. `int *temp = *a;`：把 int 赋给 int*，编译错误；应为 `int temp = *a;`
2. `*b = *temp;`：对 int 解引用，编译错误；应为 `*b = temp;`
3. 页顶红字小注“引用”与下方 `int &c=a; int &d=b;` 对应，说明作者想演示“c,d 是别名”。

笔记 487 末尾那段耐人寻味的代码：

```
int a = 10, b = 20;
swap_01(a, b);                  // a=10 b=20
swap_02(&a, &b);                // a=20 b=10
int &c = a;  int &d = b;        // c 是 a 的别名，d 是 b 的别名
b = c;                          // 即 b = a  → b = 20
a = d;                          // 即 a = b  → a = 20
// 两个变量最后都是 ！20
```

我实测，最终 a=20，b=20，因为 b=c 先把 b 改成了 20，再用已被改掉的 b（即 d）去赋给 a。这正是“引用不是副本”的最好反面教材：c,d 随 a,b 而变。

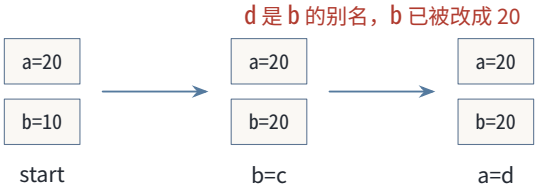


图 32　别名随原名变化：b=c 之后 d 也读到 20

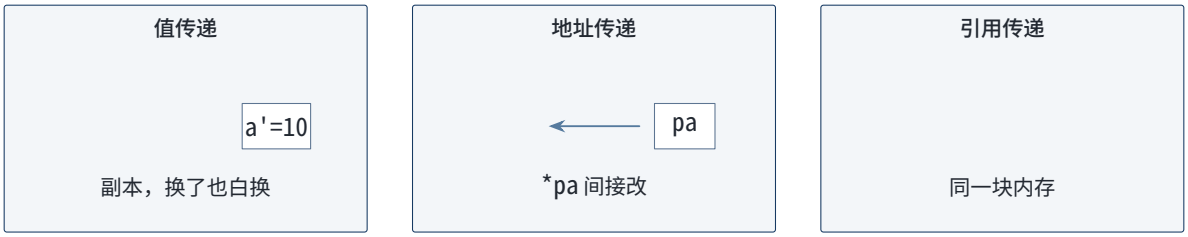


图 33　三种传参方式：副本、指针指过去、别名

结论　笔记 488 红字总结

通过引用参数产生的效果同按地址传递一样，但**语法更清楚简单**。PDF 第 31 页补充了：实参形式是变量名，但在函数调用时是将实参变量**地址**传给形参引用变量，属于**地址传递**；引用变量不单独占内存空间（节约内存空间），且直接访问。

生活中的它

地址传递像把家里的钥匙交给保洁（对方要拿钥匙开门，再进去整理——间接）；引用传递像把保洁直接请到你的房间里干活（直接）；值传递像拿着你房间的照片让保洁在照片上收拾——房间纹丝不动。

对照：值传递、地址传递、引用传递

	值传递	地址传递	引用传递
改得动实参	✘	✔	✔
调用写法	f(a)	f(&a)	f(a)
函数内写法	a	*a	a
可传空	—	✔（需检查）	✘
大对象不拷贝	✘	✔	✔

迁移判断： “值传递不影响实参”✔；“引用传递调用时不需要加 &”✔；“引用传递之后可以让形参变成另一个实参的别名”✘。

4.4　引用做函数返回值

笔记 488–489

定义　引用作为返回值

1. 作用：引用是可以作为函数的返回值存在的；
2. 注意：**不要返回局部变量引用**；

3. 用法：**函数调用可作为左值**：若函数返回值是引用，这个函数调用就可以放在赋值号左边（例：`func_01() = 2000;`）。

```
int& func() {
    int a = 10;
    return a;            // a 到此“死亡”（栈区）：返回的是已消亡变量的别名
}
int& func_01() {
    static int a = 10;   // 静态变量（全局区），函数结束后依然活着
    return a;
}
int main() {
    int &b = func();     // 笔记红字：作为引用 → 报错（见下方订正）
    int &c = func_01();  // ✔
    func_01() = 2000;    // ✔ 函数调用作为左值：把静态变量 a 改为 2000
    cout << c << endl;   // 2000
}
```

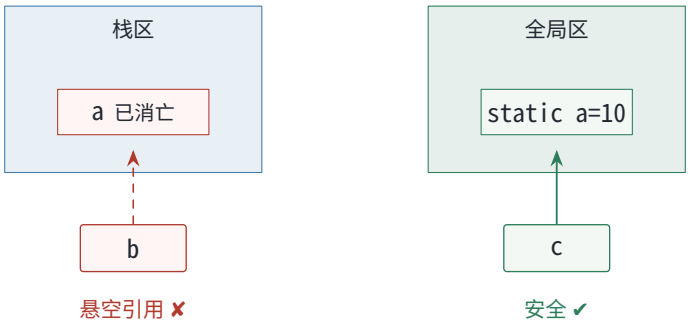


图 34　返回局部变量的引用 vs 返回静态变量的引用

原图纠错　笔记 488：“作为引用 → 报错”

`int &b = func();`（func 返回局部变量 a 的引用）在 g++ 下只给出**警告** `reference to local variable 'a' returned`，**编译通过**。真正的问题是运行期：b 所代表的对象已经消亡，读写 b 是**未定义行为**。所以“报错”准确的说法是“编译器会警告 + 运行时危险”。笔记 489 红字里的 `test_01() = 2000;` 是函数名笔误，与示例的 `func_01` 对应。

生活中的它

商场的“失物招领”广播：“请到服务台认领张三的手机”。如果张三的手机早被人取走（局部变量消亡），你到了服务台却认领成了别人的——广播（引用）仍在，但对象已没了。而挂在墙上的“营业时间牌”（static）永远在那里，随时可读可改。

对照：返回引用 与 返回值，用在赋值号左边

	int& f()	int f()
f() = 5; 合法	✔	✘（右值）
返回的是	对象本身（别名）	值的副本
可返回局部变量	✘	✔
可返回 static/全局/形参引用	✔	✔



图 35　只有返回引用，函数调用才能当左值

迁移判断：　“返回引用可以避免拷贝大对象”✔；“只要返回的是引用，什么变量都能返回”✘。

4.5 引用的本质

笔记 489；PDF 32

结论　本质

引用的本质，在 C++ 内部实现是一个**指针常量**（int *const）。因此：引用必须初始化；初始化后不可改指向；但可以改它指向的值。

```
void func(int &ref) {          // 发现是引用，转换为 int* const ref = 实参&;
    ref = 100;                 // ref 是引用，自动变为 *ref = 100;
}
int main() {
    int a = 10;
    int &b = a;                // 编译器看成 int* const b = &a;
    b = 20;                    // 编译器看成 *b = 20;
}
```

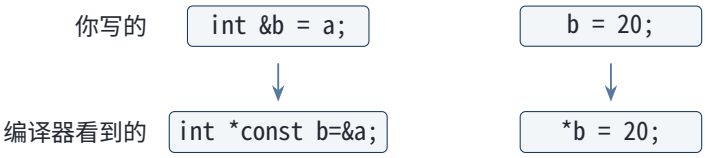


图 36　引用的语法糖：自动加上取地址与解引用

生活中的它

自动门：你走到门前，门自动识别、自动滑开（指针取地址、解引用），你不用拿钥匙——但门后面其实还是一把锁（指针常量）。

对照：引用 与 int *const

	引用	int *const p
必须初始化	✔	✔
可改指向	✘	✘
可改指向的值	✔	✔
访问值要加 *	✘（自动）	✔
可取到“指针自身”的地址	✘	✔

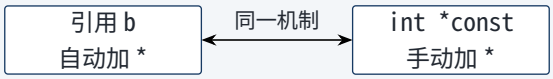


图 37　同一机制，两种写法（教学补全：语言标准只规定语义，并不规定引用一定占内存；多数编译器的实现是这样）

迁移判断： “引用是语法糖”✔；“sizeof(b) 得到指针大小”✘——sizeof 对引用返回被引用对象的大小（教学补全）。

4.6　常量引用

笔记 489–490；PDF 33–34

定义　常量引用

作用：常量引用主要用来**修饰形参，防止误操作**。方法：在函数形参列表中，可以加 const 修饰形参，防止形参改变实参。

```
void showValue(const int &b) {
    cout << b << endl;              // 笔记写成 cout << ; 应为V b
    // b = 5;                       // ✘ 不可修改
}
int main() {
    // int &ref = 10;               // ✘ 不被允许：10 是常量，没有“盒子”
    const int &ref = 10;            // ✔ 编译器自动做：int temp = 10; const int &ref = temp;
    // ref = 100;                   // ✘ 不可修改
    cout << ref << endl;
    int a = 10;
    showValue(a);                   // ✔ 大对象传参：不拷贝，也改不动
    return 0;
}
```



图 38　const int &ref = 10;：编译器偷偷准备了一个临时变量（PDF 第 34 页同理：const int &a=i+3;）

生活中的它

博物馆的展柜玻璃：游客可以近距离看（读），但不能碰（写）。给形参加 const 引用，就是把玻璃罩上——传大件展品又不用复制一份（不拷贝），也不怕被弄坏。

我实测了“大对象传参”的代价：一个 16 KB 的结构体，调用 200000 次（-O0），按值传递约 20–24 ms，const 引用传递约 0.45–0.52 ms，差距约 40–50 倍。（具体数值随机器变化，但数量级的差距稳定。）

对照：int&、const int&（以及绑定字面量）

	int &r	const int &r
绑定变量 a	✔	✔
绑定字面量 10 / 表达式 i+3	✘	✔（临时变量）
通过 r 修改	✔	✘
a 本身能否再改	✔	✔（PDF 33：i=3 合法）



图 39　常量引用把“写权限”收走；但 a 仍可经原名被修改

迁移判断： const int *p 也是“只读通道” ✔；const int &r 让 a 变成常量 ✘（只是这个别名只读）；传参用 const T& 防止误改 ✔。

4.7　PDF 补充：引用的更多规则

PDF 第 33–34 页

编号	规则	示例（判定）
1	不能建立 void 类型的引用	void &r=...; ✘
2	不能建立引用数组（数组名可以被引用，见 7）	char &rc[6]=c; ✘
3	可以将引用变量的地址赋给指针	int &b=a; int *p=&b; ✔ p 指向 a
4	可以建立指针变量的引用	int *p=&i; int *&pt=p; ✔
5	可用 const 限定引用，不允许改变该引用的值	const int &a=i; a=3; ✘ i=3; ✔
6	可用常量或表达式初始化，但必须用 const	const int &a=i+3; ✔
7	可建立数组元素的引用，也可建立数组名的引用	int (&b)[5]=a; ✔

```
int a[5] = {10, 20};
int &a_1 = a[1];                // a_1 是数组元素 a[1] 的别名
int (&b)[5] = a;                // b 是数组名 a 的别名
cout << a_1 << ", " << b[0] << endl;   // 输出 20,  (相当于输出10 a、[1]a) [0]
```

原图纠错　PDF 第 34 页：输出分隔符

课件的输出语句写成 cout<<a_1<<'、'<<b[0]<<endl;。'、' 是一个中文字符，在 C++ 里是多字节字符常量（multi-character constant），不是正常的 char，各编译器行为不同且会给警告。我实测改成 ', ' 后输出 20, 10，与课件注释一致。

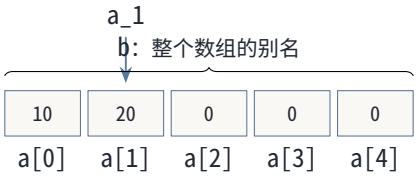


图 40　数组元素的引用 a_1 与数组名的引用 b

对照：引用数组 与 数组的引用

	int &r[5]	int (&r)[5]
含义	5 个引用组成的数组	一个引用，代表一个 5 元素数组
合法	✘	✔



图 41　括号的位置决定“引用”还是“数组”优先

迁移判断： 指针数组 `int *p[5]` 合法 ✔；数组指针 `int (*p)[5]` 合法 ✔；引用数组 ✘。

学完这里，你应该能够回答

1. `int &b=a;` 后 `b=30`，`a` 变成多少？`&a==&b` 吗？
2. 为什么引用必须初始化、且不能改绑？
3. 引用传参与指针传参各有什么好处？
4. 为什么 `func_01() = 2000;` 合法？前提是什么？
5. `const int &ref=10;` 编译器偷偷做了什么？

第5章　函数提高：默认参数、占位参数、重载与内联

来源 笔记 Memo 490–494（第三章：函数提高）；PDF 第 20–27 页（函数的重载、内联函数、默认参数）。

5.1　函数默认参数

笔记 490–491；PDF 26–27

定义　默认参数

在　C++　中，函数的形参列表中的形参是可以有**默认值**的。语法：返回值类型　函数名(参数 = 默认值)。调用时，实参个数可以与形参个数不同；**无对应实参的形参采用默认值**。

结论　两条规则

1. 若某个位置的参数有默认值，则**从这个位置往后，从左向右**，必须都有默认值（默认值只能集中放在形参表**最右端**）；
2. 若函数**声明**有默认值，函数**实现**（定义）的时候就不可以再有默认参数。

```
int sum(int a = 10, int b = 20)  { return a + b; }
int sum_2(int a, int b = 20)     { return a + b; }
```

```
int sum_3(int a = 10, int b = 20);        // 声明处给默认值
int main() {
    cout << sum()       << endl;   // 30  两个都用默认值
    cout << sum_2(5)    << endl;   // 25  b 用默认值 20
    cout << sum_3()     << endl;   // 30
    cout << sum(5, 6)   << endl;   // 11  不用默认值
}
int sum_3(int a, int b) { return a + b; }  // 定义处不再写默认值
```

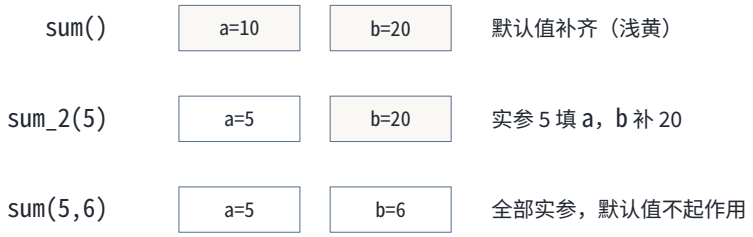


图 42　实参从左到右填满形参；剩下的由默认值补上（浅黄色为默认值）

生活中的它

咖啡店点单：“来杯拿铁”（只说一个参数），店员自动补上“中杯、热、全糖”（默认值）。你若想要大杯，就得从左往右把前面的都说出来——不可能只说“大杯”而跳过“杯型之前”的选项。

教学补全　PDF 第 26 页的四个例子

声明	判断
`float volume(float h=2.0, float r);`	✘ 默认值不在最右端
`float volume(float h, float r=12.5);`	✔
`void f1(float a, int b=0, int c, char d='a');`	✘ b 有默认值而其后 c 没有
`void f2(float a, int c, int =0, char d='a');`	✔ 声明时可不加变量名

调用 f2：f2(3.5,5,3) 最后一个取默认；f2(3.5,5) 最后两个取默认；f2(3.5) ✘ 至少要两个实参。

原图纠错　笔记 491：sum_3 的写法

笔记 491 的示例把声明写成 `int sum_3(int a, int b);`（无默认值），把默认值写在**定义**里（`int sum_3(int a=10, int b=20){...}`，位于 main 之后），并在 main 里调用 sum_3()。我在 g++ 下实测：sum_3() 报 too few arguments to function——因为调用点**只看得到声明**，看不到后面定义里的默认值。这与笔记红字规则“声明有默认值则实现不可有”正相反的写法。**正确做法**：默认值写在**声明**（或最先出现的那处）；定义里不再写。函数名大小写也要一致（Sum_3 与 sum_3 不同）。

原图纠错　PDF 第 27 页：“定义处也可给出默认值，但以声明处为准”

更严格的说法：**同一参数的默认值在同一作用域内只能给一次**。我实测在声明与定义两处都写 b=5，g++ 报 default argument given for parameter 2 of 'int f(int, int)'，所以“两处都写”会报错。课件的两个例子是“只在定义处”或“只在声明处”。

对照：默认值写在声明 与 写在定义

	声明处	定义处
有声明时	✔ 应写在这里	✘ 不能再写
无独立声明时	—	✔ 写在这里
两处都写	✘ 编译错误	

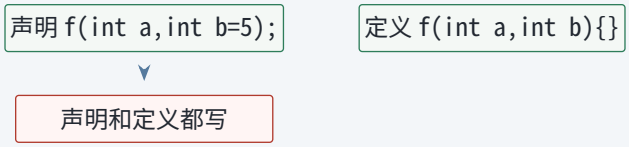


图 43　默认值只能出现一次

迁移判断： “调用点必须能看到默认值”✔；“默认值可以是表达式 / 全局变量”△ 课件未涉及，教学补全；“默认参数可以放在中间”✘。

5.2　函数占位参数

笔记 492

定义　占位参数

C++　中函数的形参列表里可以有**占位参数**，用来做占位，**调用函数时必须填补该位置**。语法：返回值类型 函数名(数据类型) {}。

```
void func(int a, int) {              // 第二个参数只有类型，没有名字
    cout << "这是函数" << endl;
}
int main() { func(10, 20); }         // 必须传两个实参
// 笔记红字：占位参数也有默认参数
void func2(int a, int = 10) { }      // 实测：func2(1) 与 func2(1,2) 都合法
```

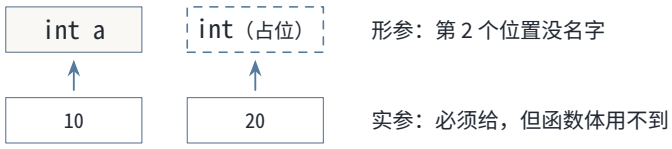


图 44　占位参数：占着位置，不能省略（除非它有默认值）

为什么需要它

占位参数在现阶段用得少；它的典型用途是**预留接口**（以后想加参数而不破坏旧调用），以及在后面学运算符重载时区分前置/后置 ++（operator++(int)，课程之后会遇到）。

生活中的它

电影院的“预留座位”：位置在那里，现在没人坐（函数体没有用到），但订票时必须登记这个座位（调用时要填）。

对照：占位参数 与 默认参数

	占位参数 int	默认参数 int b=10
调用时必须传	✔	✘
函数体里能用	✘（没名字）	✔
可同时具备	int = 10：既占位又有默认值	



图 45　三种形参形态

迁移判断： “占位参数也可以有默认值”✔（笔记红字）；“占位参数能在函数体里使用”✘。

5.3 函数重载

笔记 492–494；PDF 20–22, 27

定义　函数重载

1. **作用**：函数名可以相同，提高复用性。
2. **条件**：① 同一个作用域下；② 函数名称相同；③ 函数**参数类型不同**，或**个数不同**，或**顺序不同**。
3. **注**：函数的**返回值不可以作为函数重载的条件**。

笔记 492–494 的示例：

```
void func()                     { cout << "func()" << endl; }
void func(int a)                { cout << "func(int a)" << endl; }
void func(double a)             { cout << "func(double a)" << endl; }
void func(int a, double b)      { cout << "func(int a, double b)" << endl; }
void func(double a, int b)      { cout << "func(double a, int b)" << endl; }
int main() {
    func();          // func()
    func(10);        // func(int a)
    func(3.14);      // func(double a)
    func(5, 3.14);   // func(int a, double b)
    func(3.14, 5);   // func(double a, int b)
}
```

上面的输出是我逐行实测验证过的。PDF 第 21 页的例子：同名函数 max，按**参数个数**不同重载（2 个整数或 3 个整数）；PDF 第 20 页则按**参数类型**不同重载（int 与 double）。

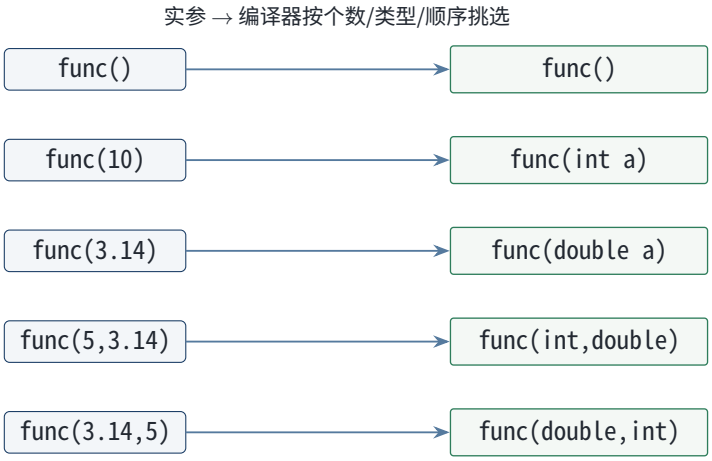


图 46　重载决议：调用点的实参决定走哪一个函数

生活中的它

“吃”这个动词：吃饭、吃面、吃药，名字相同，对象不同，动作也不同。你不会为每种“吃”发明一个新动词，而是通过后面的宾语（参数）判断是哪种吃。

教学补全　PDF 第 22 页：什么算“不同”

两个函数	重载?	原因
`void f(int a, float b)` 与 `void f(float b, int a)`	✔	参数顺序不同
`int f(int a)` 与 `void f(int b)`	✘	只有返回类型/形参名不同，编译不通过

并且 PDF 提醒：使用重载时，同名函数的功能应当相同或相近，否则可读性不好。

原图纠错　PDF 第 20 页标题

课件第 20 页的标题写的是“掌握函数的重载、函数的模版、内置函数、函数的默认值参数”，但随后的内容并没有讲函数模板，且“内置函数”即后面所说的“内联函数”。模板属于后续实验（实验七），本讲义不展开。

对照：函数重载 与 函数默认参数 与 不同函数名

	重载	默认参数	不同名
同名	✔	✔	✘
函数体数量	多个	一个	多个
行为可以不同	✔	✘	✔
调用方式	按参数匹配	少写参数	记不同名字

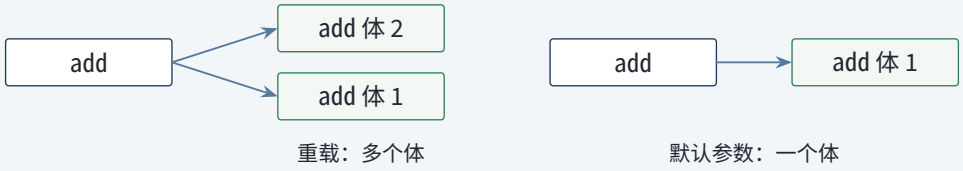


图 47　一个名字对应多个函数体 vs 一个函数体多种调用写法

> **迁移判断：** 二者都让调用写法更灵活 ✔；**同时使用**可能引起二义性（见下）△；返回类型不同就算重载 ✘。

注意事项（笔记 494）

种类： ① 引用作为重载条件；② 函数重载碰到函数默认参数（红字：② **情况要避免**）。

```
void func(int &a)       { cout << "func(int &a)" << endl; }
void func(const int &a) { cout << "func(const int &a)" << endl; }
int main() {
    int a = 10;
    func(a);     // func(int &a)        a 是变量，能绑定 int&
    func(10);    // func(const int &a)  10 是字面量，只能绑定 const int&
}
```

我实测的输出与注释一致。

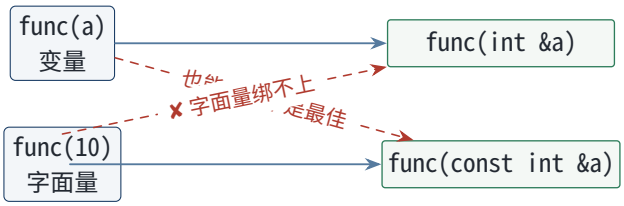


图 48　int& 与 const int& 作重载：变量优先非常量版本，字面量只能走常量版本

② **默认参数造成二义性（PDF 第 27 页）：** 一个函数不能既作为重载函数，又作为有默认参数的函数，否则会出现**二义性**，系统无法执行。

```
void add(int a, int b, int c = 0) { cout << "a"; }
void add(int a, int b)            { cout << "b"; }
int main() { add(2, 2); }     // ✘ 两个都能匹配：ambiguous
```

我实测 g++ 报错：`call of overloaded 'add(int, int)' is ambiguous`。

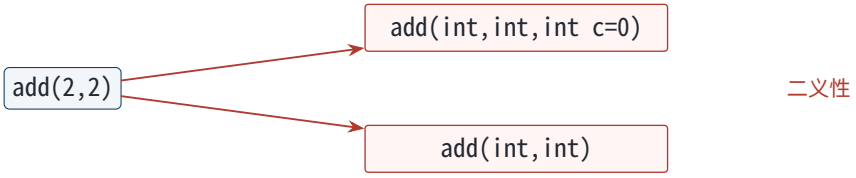


图 49　同一次调用匹配两个候选，编译器无法取舍

原图纠错　重载里的“类型转换”陷阱——教学补全

我在上例的 `func(int)` / `func(double)` 重载基础上实测：`func(3.14f)`（float）匹配的是 `func(double)`（提升），`func('a')`（char）匹配 `func(int)`（提升）。而 `func(5L)`（long）会报二义性——因为 long→int 与 long→double 同级。规则很多，原则是：**尽量让参数类型一一对应，避免依赖隐式转换**。

生活中的它

同一家饭店菜单上写了“炒饭（可加蛋）”，又单独列了“炒饭”。你点“炒饭”，服务员要问：“您是要普通炒饭，还是不加蛋的‘加蛋炒饭’？”这就是二义性。

对照：重载条件对照：哪些真的算“不同”

差异	能否构成重载
参数个数不同	✔
参数类型不同（int / double）	✔
参数顺序不同（类型不同时）	✔
int& 与 const int&	✔
int 与 const int（按值传参）	✘（const 不影响按值参数的类型）
返回类型不同	✘
形参名不同	✘



图 50　绿色：能区分重载；红色：不能

迁移判断： “只有返回类型不同不是重载”✔；“在不同作用域（如不同的类、不同函数内）同名也叫重载”✘。

5.4 内联函数

PDF 第 23–25 页

定义　内联函数 inline

C++ 提供一种提高效率的方法：**内联函数**。定义方法：在函数类型前加上 inline。在**编译时**将所调用的内联函数的代码直接**嵌入**到主调函数中，而不是将流程转出去/转回来。调用函数时需要一定的**时间开销**（保存现场、跳转、返回）。 实质：**用存储空间（使用更多的存储空间）来换取时间（减少执行时间）**。

```
inline int max(int, int, int);          // 声明内联函数，注意左端有 inline
int main() {
    int i = 10, j = 20, k = 30, m;
    m = max(i, j, k);                   // 编译时将 max 代码嵌入进来
    cout << "max=" << m << endl;        // max=30
}
inline int max(int a, int b, int c) {   // 定义 max 为内联函数
    if (b > a) a = b;
    if (c > a) a = c;
    return a;                           // 返回 a, b, c 中的最大值
}
```

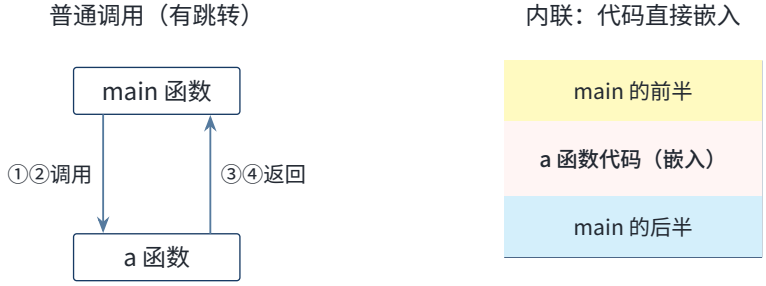


图 51　普通函数调用要“出去再回来”；内联函数直接把函数体复制进调用处（PDF 第 23 页）

教学补全　PDF 第 25 页：注意事项

- 可以在声明函数和定义函数时同时写 inline，也可以只在其中一处声明 inline，只要保证在**调用函数之前**将 inline 信息告知编译系统。
- 一般只将规模很小（一般为 5 个语句以下）而使用频繁的函数声明为内置函数。
- 内置函数中不能包括复杂的控制语句，如循环语句和 switch 语句。
- 对函数作 inline 声明，只是对编译系统提出的一个**建议**，而**不是命令**；编译系统会根据具体情况决定是否按内联函数来处理。

我实测验证了“建议而不是命令”：同一个 inline 函数，g++ -O0 生成的汇编里仍有一条 call 指向它；-O2 则把它完全内联（call 数为 0）。

原图纠错　现代 C++ 的补充

“不能包含循环和 switch”是旧编译器的限制；现代编译器允许，只是不一定内联。另外，在今天的 C++ 里 inline 更重要的意义是“允许同一个函数定义出现在多个源文件（头文件）里而不违反单一定义规则”。

生活中的它

炒菜时每次都跑去隔壁房间拿盐（函数调用），不如在灶台旁边直接放一小罐盐（内联）：省了来回的时间，代价是每个灶台都得放一罐（占更多空间）。

对照：inline 函数 与 普通函数 与 宏函数

	普通函数	inline 函数	宏 #define
有调用开销	✔	通常没有	没有
有参数类型检查	✔	✔	✘
代码体积	小	可能变大	可能变大
调试容易	✔	大致 ✔	✘
是否必然展开	—	✘（建议）	✔

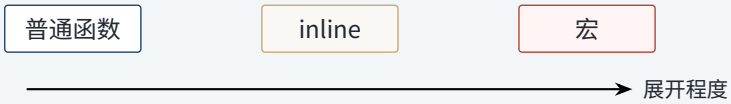


图 52　从“从不展开”到“一定展开”的光谱

迁移判断：“内联以空间换时间”✔；“所有小函数都应当 inline”✘；“递归函数一定能被内联”✘。

学完这里，你应该能够回答

1. 为什么默认参数要放在形参表最右端？默认值应写在声明还是定义？
2. 占位参数在调用时可以省略吗？它有什么用？
3. 函数重载的三个条件是什么？返回类型为什么不能作为条件？
4. func(int&) 与 func(const int&) 谁处理 func(10)？
5. inline 是命令还是建议？它的实质是什么交换？

第 6 章　字符串类型 string

来源 PDF 第 35–38 页（字符串变量）。笔记 Memo 482 的 `string str="Hello, World!"` 也与本章有关。

6.1　为什么需要 string

定义　string 类

用**字符数组**来存放字符串并不是最理想、最安全的方法。C++ 提供了一种新的数据类型——**字符串类型（string 类）**，用来定义字符串变量。string 并不是 C++ 的基本类型，它是在 C++ 标准库中声明的一个**字符串类**，每一个字符串变量都是 string 类的一个**对象**。使用它必须包含头文件 `#include <string>`（**注意：头文件名不是 string.h**）并 `using namespace std;`。

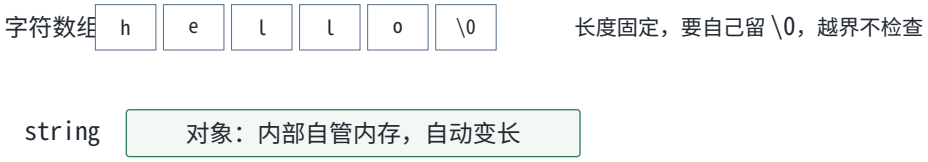


图 53　字符数组需要你操心的事，string 替你操心了

生活中的它

手写便签（字符数组）要自己留够空白，写满了就写不下；而电子笔记本（string）会自动换页、自动记长度。

对照：字符数组 与 string

	char s[10]	string s
长度固定	✔	✘
赋值 s2=s1	✘（要 strcpy）	✔
连接	strcat	+
比较	strcmp	== < >
越界保护	✘	△（at() 检查，[] 不检查）



图 54　谁来管理内存

迁移判断：　“string 对象可用 == 比较”✔；“char* 指针也可用 == 比较内容”✘（比的是地址）。

6.2　定义、赋值与访问

PDF 第 35–36 页

```
string string1, string2;                    // 定义字符串变量（不需要指定长度）
string2 = "I am sure I will be successful";   // 字符串常量赋给字符串变量
string2 = string1;                          // 将 string1 复制一份给 string2
string word = "Then";
word[2] = 'a';                              // 可对某一字符操作：word 变为 "Than"
```

```
cout << sizeof(string);                    // PDF 写 ，实测见下16
```

原图纠错　PDF 第 35 页：sizeof(string) 与“存放字符串地址”

课件写“在 Visual C++ 中为每一个字符串变量分配 16B；sizeof(string) 输出 16，与平台有关（Dev 8）”，并说“字符串变量中存放字符串的地址，而不是字符串本身”。我在 g++ 13（64 位 Linux）上实测 sizeof(string) 为 32。它之所以与平台有关，是因为 string 对象内部除了指向字符的指针外，还有长度、容量，以及一小块用来存短字符串的缓冲区（短字符串优化 SSO）。所以“只存地址”对长字符串大致成立，对短字符串（如 "Hello, World!"，13 个字符）则字符就在对象内部——我在第 3 章的实测中看到 str.data() 恰在 &str 之后 16 字节处。



短字符串：字符就存在缓冲里；长字符串：ptr 指向堆上的字符

图 55　libstdc++ 的 std::string 结构示意（实现相关；不同编译器大小不同）

生活中的它

行李箱的外袋：小物件（短字符串）直接塞进外袋，不用开大箱；东西多了，才使用后备箱（堆）并在外袋放一张“后备箱位置卡”（指针）。

对照：string 的 s[i] 与 at(i)（教学补全）

	s[i]	s.at(i)
越界检查	✘	✔（抛 out_of_range）
速度	快	略慢

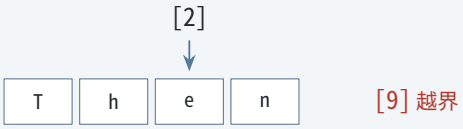


图 56　下标越界

迁移判断： 数组下标越界不检查 ✔；string::operator[] 同样不检查 ✔。

6.3　输入输出与运算

PDF 第 36–38 页

```
string s1;
cin >> s1;                 // 输入一个不含空格的字符串给 （遇空白停止）s1
getline(cin, s1);          // 输入一行（可含空格）给 s1
cout << s1;                // 输出整个字符串
cout << s1[0];             // 输出一个字符
// 运算
string a = "C++ ", b = "Language";
a = a + b;                 // 连接，等于 ；strcata 变为 "C++ Language"
string t1, t2, temp;
```

```
cin >> t1 >> t2;          // 输入 today china
if (t1 > t2) { temp = t1; t1 = t2; t2 = temp; }    // 按字典序交换
cout << t1 << " " << t2 << endl;    // china today
```

字符串复制用赋值运算符（string1=string2;，同 strcpy）；**连接**用加号（同 strcat）；**比较**直接用关系运算符 > < == !=（PDF 第 37–38 页）。

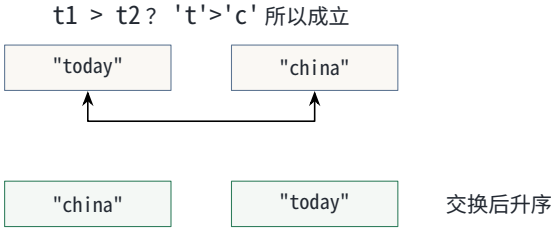


图 57　字符串按字典序比较：先比第一个字符；"today" > "china"

生活中的它

字典里“china”排在“today”前面，因为 c 在 t 前。比较字符串，就是“翻字典”比大小。

对照：cin >> s 与 getline(cin, s)

输入 "hello world"	cin >> s	getline(cin,s)
s 的内容	"hello"	"hello world"
会跳过前导空白	✔	✘
遇空格停止	✔	✘（遇换行才停）



图 58　两种读法读到的范围

迁移判断： cin >> a >> b 连读两个单词 ✔；cin >> n 之后紧接 getline，会先读到残留的换行 △ 课件未提，教学补全。

学完这里，你应该能够回答

1. 为什么 #include <string> 而不是 <string.h>?
2. word[2]='a' 之后 "Then" 变成什么?
3. cin >> s 与 getline 的区别?
4. "today" > "china" 为真吗？为什么?

第 7 章　综合：把内存、引用与函数串起来

来源 综合笔记 481–494 与课件 PDF 第 5–40 页的全部内容。

7.1 一张概念地图

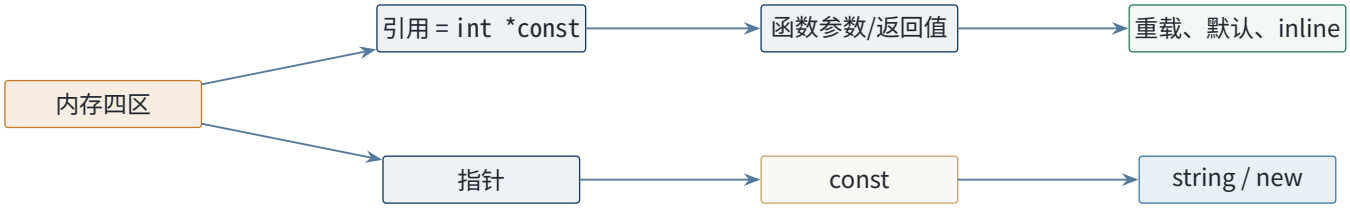


图 59　全讲义概念地图：内存四区是底座，指针/引用是工具，函数和 const 是用法

7.2 函数怎样把结果交出来：四种方式对照

返回方式	写法	数据在哪	安全?
返回值	`int f(){int a=10; return a;}`	拷贝到调用者	✔
返回局部地址	`int* f(){int a; return &a;}`	栈（已消亡）	✘
返回局部引用	`int& f(){int a; return a;}`	栈（已消亡）	✘
返回 static 引用	`int& f(){static int a; return a;}`	全局区	✔
返回 new 指针	`int* f(){return new int(10);}`	堆区	✔（要 delete）

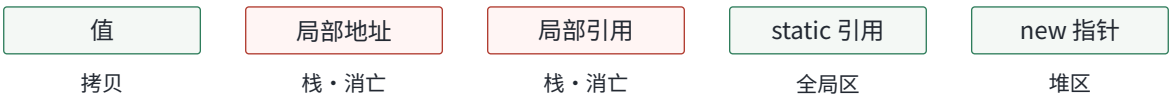


图 60　五种“交出结果”方式的存活情况（绿色安全，红色危险）

生活中的它

外卖送达有五种方式：把饭菜送到手上（值）；只给你一个已拆除餐厅的地址（局部地址/引用）；放在大厦大堂的长期保温柜（static）；放在租来的储物间并给你钥匙（new）。

7.3 综合示例：把本讲义用一遍

例题　完整小程序

```
#include <iostream>
#include <string>
using namespace std;
const int LIMIT = 100;                          // 全局常量（全局区）
inline int clamp(int v) { return v > LIMIT ? LIMIT : v; }   // 内联
void swap2(int &x, int &y) { int t = x; x = y; y = t; }     // 引用传参
int sum(int a, int b = 20) { return a + b; }                // 默认参数
int sum(int a, int b, int c) { return a + b + c; }          // 重载
void show(const string &s) { cout << s << endl; }           // 常量引用，不拷贝
int main() {
    int a = 5, b = 8;
    swap2(a, b);                          // a=8 b=5
    int *p = new int(sum(a));             // 堆区; sum(8) = 28
    show("p 指向 " + to_string(*p));      // 输出 p 指向 28
    cout << clamp(*p + 200) << endl;      // 100
    delete p;                             // 释放
```

```
}
```

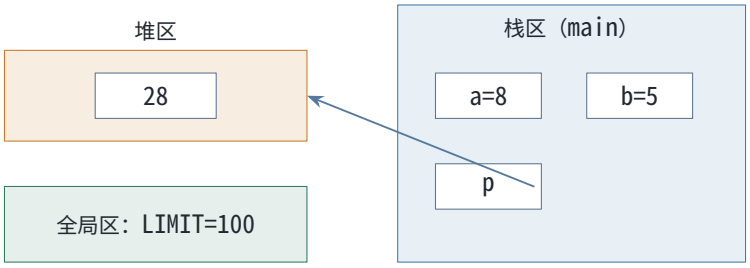


图 61　综合示例运行到 delete 之前的内存快照

7.4　从课本到现实：为什么这些细节值钱

从课本到现实

下面的数字都来自我在这个环境里的实测（Linux，g++ 13）。

- **栈很小，堆很大。** `ulimit -s` 显示栈上限为 8192 KB（8 MB）。一个无限递归的函数很快把栈耗光，运行时报段错误（实测退出码 139）；在栈上定义 `int big[10000000]`（约 40 MB）同样段错误，而用 `new int[10000000]` 顺利完成。
- **传大对象，引用快得多。** 一个 16 KB 的结构体调用 200000 次：按值约 20–24 ms，`const` 引用约 0.45–0.52 ms，差约 40–50 倍。这也是现代 C++ 规范（C++ Core Guidelines）建议“较大的对象用 `const T&` 传参”的原因。
- **忘记 `delete` 是真实泄漏。** 带 AddressSanitizer 的程序在退出时直接列出泄漏的块；每次 `new` 得到不同地址，说明内存确实被占用。服务器长期运行的程序，几字节的泄漏也会累积成故障。
- **地址每次不同，顺序不变。** 开启地址空间布局随机化（ASLR）后，同一个程序多次运行的地址前缀不同（见图 22），但“代码 < 常量 < 全局 < 堆 < 栈”的次序稳定。

工程里，人们尽量不手写 `new/delete`，而用 `vector`、`unique_ptr` 等自动释放的“智能容器”（课外）。但它们的内部，仍然是你在第 3 章学的堆与栈。

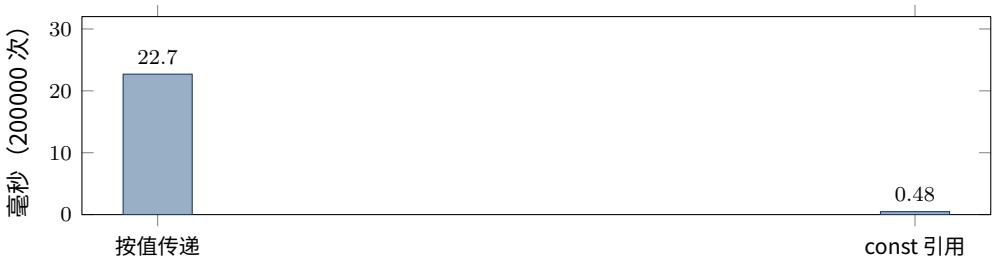


图 62　16 KB 结构体传参耗时对比（实测一次：22.7 ms vs 0.48 ms，-O0）

小结

- 内存四区：代码区（只读指令）、全局区（全局/静态/常量）、栈区（自动）、堆区（手动）。
- 引用 = 别名 = `int *const`：必须初始化、不能改绑、可当左值、可做 `const` 保护。
- 函数提高：默认参数靠右、占位参数要填、重载看参数不看返回值、`inline` 只是建议。
- `string`：自带内存管理、可 `+ = <`、`getline` 读一行。

· new/delete 成对、new[]/delete[] 成对。

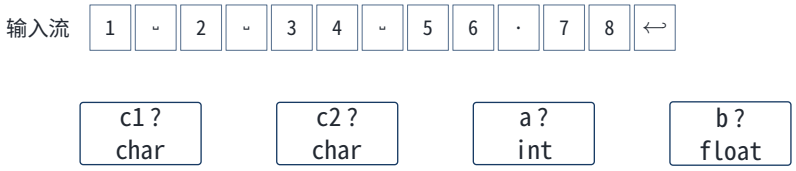
第 8 章　正式练习：38 题

定义　做题须知

本章共 38 道正式练习：基础巩固 25 题（单选 10、多选 5、填空 5、解答 5），中等偏下提升 10 题（单选 3、多选 2、填空 3、综合 2），跨学科迁移 3 题（单选 1、多选 1、填空 1）。**题目图只画已知条件，不暗示答案**；答案速查和逐题图解解析集中放在本章之后。运行环境默认：64 位 Linux，g++，int 为 4 字节、指针为 8 字节。

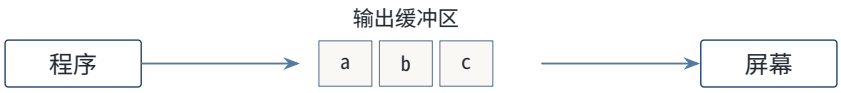
A 基础巩固训练（25 题）

第 1 题 [基础 · 单选] 在程序中执行 `char c1,c2; int a; float b; cin>>c1>>c2>>a>>b;`，键盘输入 `1 2 34 56.78`（以回车结束），则 a 的值是：



A. 2
B. 34
C. 12
D. 234

第 2 题 [基础 · 单选] 关于 cout 与输出缓冲区，下列说法正确的是：



A. '\n' 与 endl 完全等价，都会立即刷新缓冲区

B. 数据先放入缓冲区，直到缓冲区满或遇到 endl/cout.flush() 才一起输出

C. cout<<a,b,c; 会依次输出 a、b、c

D. cout 没有缓冲区，每次 << 立即显示

第 3 题 [基础 · 单选] 语句 cout<<setw(6)<<18<<'|'<<18; 的输出（␣ 表示空格，每格一个字符）是：

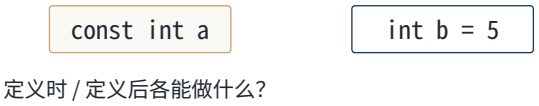
A. 

B. 

C. 

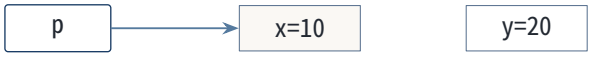
D. 

第 4 题 [基础 · 单选] 已有 `int b=5;`，下列语句中能通过编译的是：



A. `const int a;`
B. `const int a = 2 + b;`
C. `const int a = 3; a = 4;`
D. `const int a; a = 3;`

第 5 题 [基础 · 单选] 已有 `int x=10, y=20; const int *p = &x;`，下列语句中合法的是：



A. `*p = 30;`
B. `p = &y;`
C. `p = 30;`
D. `&p = &y;`

第 6 题 [基础 · 单选] 在 `main` 函数内定义 `static int s = 17;`，变量 `s` 存放在：



A. 代码区
B. 全局区
C. 栈区
D. 堆区

第 7 题 [基础 · 单选] 执行 `int* p = new int(10);` 后，指针变量 `p` 本身与它所指的数据分别存放在：

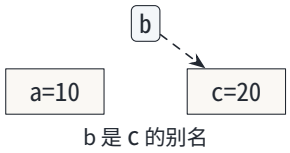


A. 栈区、栈区

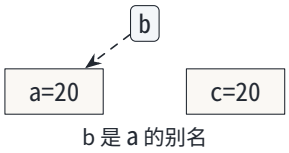
B. 栈区、堆区
C. 堆区、栈区
D. 堆区、堆区

第 8 题 [基础 · 单选] 执行 `int a=10; int &b=a; int c=20; b=c;` 之后，内存状态是（b 的连线表示它是谁的别名）：

A.

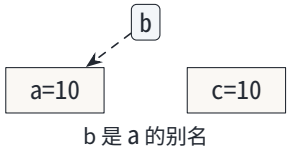


B.



C. 编译错误 ×

D.



第 9 题 [基础 · 单选] 下列各组函数中，能够构成合法函数重载的是：

`void f(  )`　签名 = 函数名 + 参数表（个数、类型、顺序）

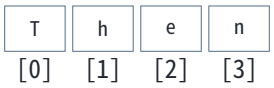
A. `int f(int a);` 与 `void f(int a);`

B. `void f(int a);` 与 `void f(int b);`

C. `void f(int a, double b);` 与 `void f(double a, int b);`

D. `void f(int a);` 与 `int f(int a);`

第 10 题 [基础 · 单选] 执行 `string s="Then"; s[2]='a'; s = s + "!"; cout<<s;`，输出是：



A. Then!
B. Than!
C. Than
D. 编译错误

第 11 题 [基础 · 多选] 关于 cin、cout，下列说法正确的有：



A. `cin>>a>>b;` 可以分写成多行

B. `cin>>a, b;` 能正确读入 a 和 b

C. 用 `cin>>` 读 `char` 时，空格、回车、制表符会被跳过

D. `endl` 除换行外还会刷新输出缓冲区

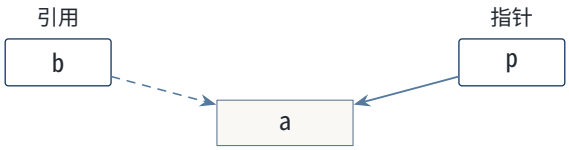
第 12 题 [基础 · 多选] 阅读下面代码，下列变量中**存放在全局区**的有：

```
int g = 20;                  // 全局变量
const int G = 30;            // const 全局常量
int main() {
    const int c = 21;        // const 局部常量
    int *p = new int(4);     // 堆上的 int
}
```



A. g
B. G
C. c
D. *p（p 所指的 int）

第 13 题 [基础 · 多选] 关于 C++ 的引用，下列说法正确的有：



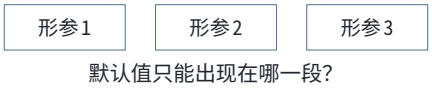
A. 引用必须在定义时初始化

B. 引用初始化后可以改为另一个变量的别名

C. 引用可以作为函数的返回值

D. 引用的本质是指针常量（`int *const`）

第 14 题 [基础 · 多选] 下列函数**声明**中，符合“默认参数必须靠右”规则的有：

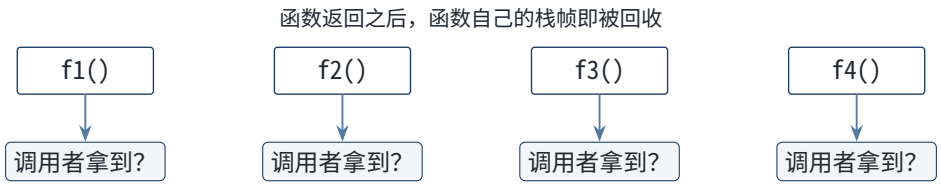


A. `void f(int a, int b = 2);`
B. `void f(int a = 1, int b);`

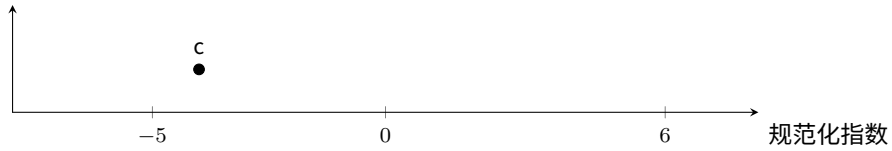
C. `void f(int a, int = 3);`
D. `void f(int a = 1, int b = 2, int c);`

第 15 题 [基础 · 多选] 下列函数写法中，返回结果在调用之后**仍然安全可用**的有：

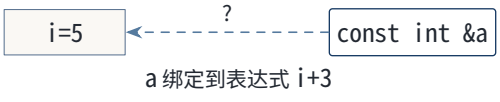
```
A:  int  f1() { int a = 1; return a; }
B:  int* f2() { int a = 2; return &a; }
C:  int& f3() { static int a = 3; return a; }
D:  int* f4() { return new int(4); }
```



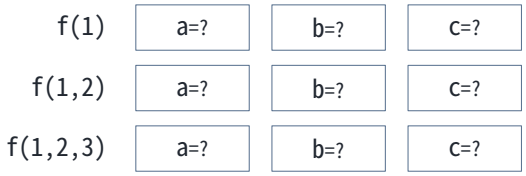
第 16 题 [基础 · 填空] 执行 `double c = 34.5678e-5; cout<<c;` 的输出为 _____。（默认格式，最多 6 位有效数字）



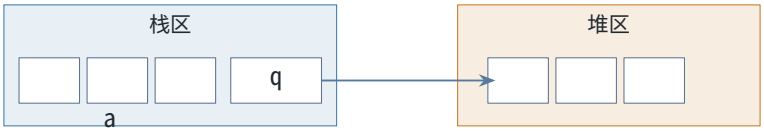
第 17 题 [基础 · 填空] 执行 `int i=5; const int &a = i+3; i = 10; cout<<a;` 的输出为 _____。



第 18 题 [基础 · 填空] 已知 `int f(int a, int b=4, int c=6) { return a+b+c; }`，则 `f(1)`=_____，`f(1,2)`=_____，`f(1,2,3)`=_____。



第 19 题　[基础 · 填空] 在 64 位系统上（int 4 字节），`int a[3]; int* q = new int[3];`，则 sizeof(a)=_____，sizeof(q)=_____。



第 20 题 [基础 · 填空] 执行下列代码，输出为 _____。（swap 为 std::swap）

```
string s1 = "today", s2 = "china";
if (s1 > s2) swap(s1, s2);
cout << s1 << " " << s2;
```



第 21 题 [基础 · 解答] 阅读代码，说明变量 g，G，x，s，p，*p，c 各自存放在哪个区，并指出哪些变量需要程序员手动释放。

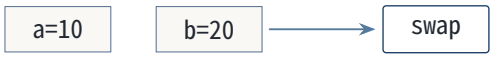
```
int g = 1;
const int G = 2;
void f(int x) {
    static int s = 3;
    int *p = new int(4);
    const int c = 5;
    delete p;
}
```



g　G　x　s　p　*p　c　（待放入四个区之一）

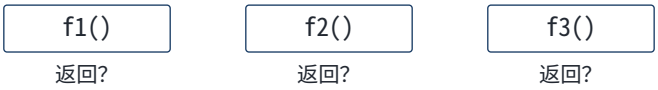
第 22 题 [基础 · 解答] 某同学写了下面的交换函数，编译报错。指出错误、写出正确版本，并分别写出用**指针传参**与**引用传参**调用后，a=10，b=20 的结果。

```
void swap_02(int *a, int *b) {
    int * temp = *a;
    *a = *b;
    *b = *temp;
}
```



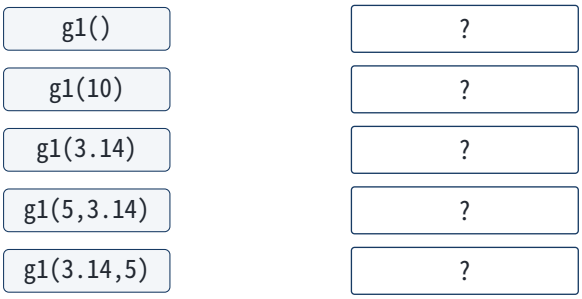
第 23 题 [基础 · 解答] 阅读代码：（1）指出哪些函数的返回结果在调用后仍然安全；（2）执行 `f3() = 30; cout << f3();` 的输出是什么？为什么这条赋值合法？

```
int  f1() { int a = 1; return a; }
int& f2() { int a = 2; return a; }
int& f3() { static int a = 3; return a; }
```

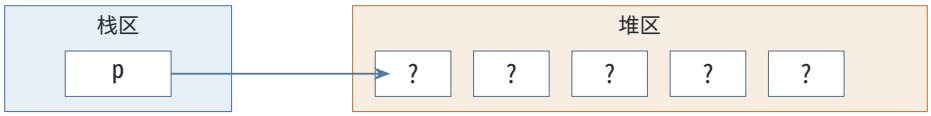


第 24 题 [基础 · 解答] 已定义下列五个同名函数，写出每个调用实际执行的函数（写出其参数表即可）。

```
void g1();              void g1(int a);       void g1(double a);
void g1(int a, double b);                     void g1(double a, int b);
// 调用：  g1();  g1(10);  g1(3.14);  g1(5, 3.14);  g1(3.14, 5);
```



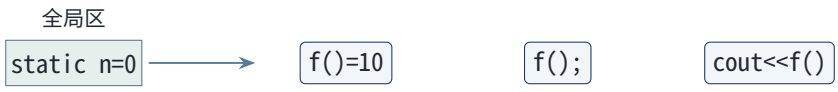
第 25 题 [基础 · 解答] 写出 C++ 代码：在堆区开辟含 5 个 int 的数组，令第 i 个元素等于 i*i，输出这 5 个数之和，并正确释放内存。并说明指针与数组各自在哪个区。



B 中等偏下提升训练（10 题）

第 26 题 [提升 · 单选] 阅读代码，输出是：

```
int& f() { static int n = 0; return ++n; }
int main() { f() = 10; f(); cout << f(); }
```



A. 3
B. 10
C. 11
D. 12

第 27 题 [提升 · 单选] 已有声明 `void h(int); void h(int, int = 5);`，下列调用中，编译错误的原因是“二义性（ambiguous）”的是：



A. `h(1, 2);`
B. `h(1);`
C. `h(1, 2, 3);`
D. `h();`

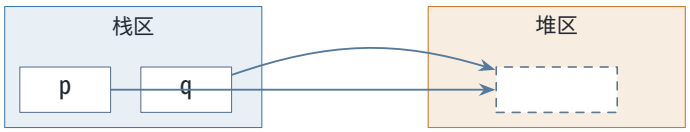
第 28 题 [提升 · 单选] 已有 `void show(const int&); void bump(int&); int a=5;`，下列调用中**不能通过编译**的是：



A. `show(10);`
B. `show(a+1);`
C. `bump(a);`
D. `bump(a+1);`

第 29 题 [提升 · 多选] 阅读代码，执行到最后一行之后，下列判断正确的有：

```
int *p = new int(10);
int *q = p;
delete p;
```



A. `q` 成为悬空指针

B. 此时通过 `*q` 读取数据是安全的，仍为 10

C. 为避免误用，宜在 `delete p;` 之后执行 `p = nullptr;`（`q` 也要处理）

D. 这段代码发生了内存泄漏

第 30 题 [提升 · 多选] 已有 `int a[5]={10,20}; int i=5; int *p=&i; char c[6]="hello";`，下列声明合法的有：



A. `int *&pt = p;`
B. `int (&b)[5] = a;`

C. `char &rc[6] = c;`
D. `const int &r = i + 3;`

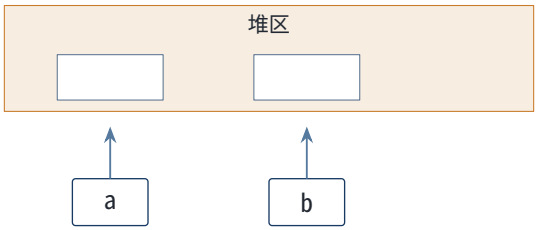
第 31 题　[提升 · 填空]　在包含 `<iomanip>` 的前提下，执行下列语句，输出为 _____（注意各空格；用 _ 表示一个空格亦可）。

```
cout << hex << 255 << ' ' << oct << 8 << ' ' << dec << 10
     << ' ' << setw(4) << 7 << '|' << 7;
```



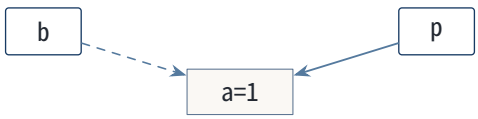
第 32 题 [提升 · 填空] 执行下列程序，cout 输出（紧挨着，无空格）为 _____；程序结束时有 _____ 块堆内存未被释放。

```
int* f() { int *p = new int(7); *p += 1; return p; }
int main() {
    int *a = f(); int *b = f();
    cout << (a == b) << *a + *b;
}
```



第 33 题 [提升 · 填空] 执行下列语句，输出为 _____（两个值之间有一个空格）。

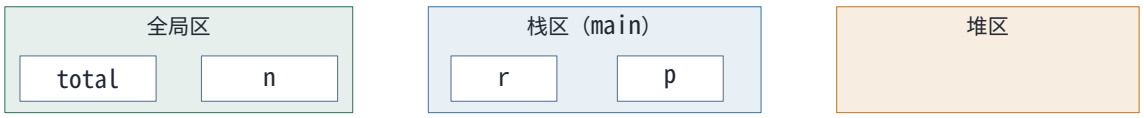
```
int a = 1;
int &b = a;
int *const p = &a;
b = b + 1;
*p = *p + 2;
cout << a << ' ' << (&a == &b);
```



第 34 题 [提升 · 综合] 阅读程序，回答（1）–（3）。

```
int total = 0;
int& bump(int step = 1) { static int n = 0; n += step; total += step; return n; }
void fill(int *a, int len, int v) { for (int i = 0; i < len; i++) a[i] = v + i; }
int main() {
```

```
    bump() = 10;
    bump(5);
    int &r = bump(0);
    r = 100;
    int *p = new int[3];
    fill(p, 3, r);
    cout << bump(0) << ' ' << total << ' ' << p[2] << endl;
    delete[] p;
}
```



（1）写出程序输出；（2）指出 total、n、r、p、p[2] 分别位于哪个区（r 说明它是谁的别名）；（3）若把 static int n = 0; 中的 static 去掉，bump(0) 的返回值会出什么问题？

第 35 题 [提升 · 综合] 阅读程序，回答（1）–（3）。

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;
int main() {
    const int N = 2;
    string name[N]; double score[N];
    for (int i = 0; i < N; i++) cin >> name[i] >> score[i];
    cout << fixed << setprecision(1);
    for (int i = 0; i < N; i++)
        cout << left << setw(6) << name[i] << right << setw(6) << score[i] << endl;
}
```



（1）输入如上（Tom 85.26 与 Amy 92 各占一行）时，写出两行输出（用 _ 表示空格）；（2）N 为什么写成 const int，能否在程序中途改成 3？（3）若第一行输入变成 Tom Lee 85.26，会发生什么？应如何修改读入方式？

C 跨学科迁移训练（3 题）

第 36 题 [跨学科 · 单选] （**嵌入式系统**）某单片机的数据 RAM 共 2048 B；本程序已用去全局区与堆区，余下留给栈的空间最多 **512 B**。栈由编译器自动管理，局部变量放在栈上。现有函数 void sample() { char buf[600]; …}（其余局部变量可忽略）。调用 sample() 时最可能发生：



数据 RAM 2048 B（示意，不按比例）

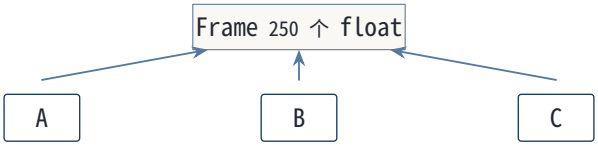
A. 编译期必然报错，因为数组不能超过 512 B

B. 运行时栈空间不足，可能发生栈溢出

C. 编译器自动把 `buf` 改放堆区，程序正常

D. `buf` 被放进全局区，程序正常

第 37 题 [跨学科 · 多选] （**测量与信号处理**） 一帧传感器数据定义为 `struct Frame { float v[250]; };`（`float` 为 4 字节）。函数 `void A(Frame f)` 按值传参，`void B(const Frame &f)` 按常量引用传参，`void C(Frame &f)` 按引用传参。下列说法正确的有：



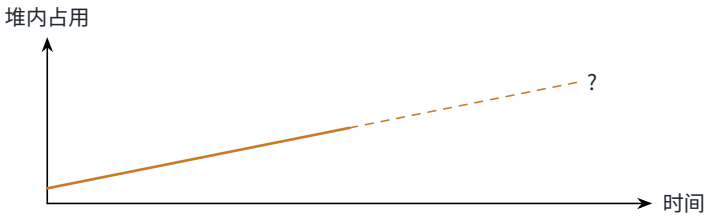
A. 每调用一次 `A(f)`，要拷贝约 1000 字节的数据

B. `B(f)` 不拷贝数据，并且函数内不能修改 `f.v[0]`

C. 在 `A` 内修改 `f.v[0]` 会改变调用者手里的那一帧

D. 在 `C` 内修改 `f.v[0]` 会改变调用者手里的那一帧

第 38 题 [跨学科 · 填空] （**计算机系统运维 / 可靠性**） 某服务器程序每处理一个请求就 `new` 一个 64 字节的对象，但忘记 `delete`。若每秒处理 1000 个请求，连续运行 1 小时，共泄漏约 _____ MB。（取 $1\ \text{MB} = 10^6\ \text{B}$，结果保留 1 位小数）



第 9 章　答案速查与逐题图解解析

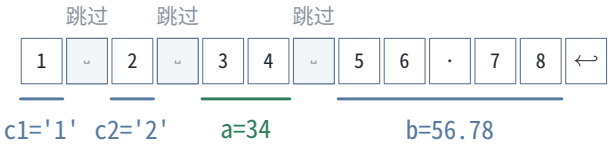
答案速查（只查结果，过程见后面的解析）

基础单选	1. B　2. B　3. B　4. B　5. B　6. B　7. B　8. B　9. C　10. B
基础多选	11. ACD　12. AB　13. ACD　14. AC　15. ACD
基础填空	16. `0.000345678`　17. `8`　18. `11`、`9`、`6`　19. `12`、`8`　20. `china today`
基础解答	21. `g`、`G`、`s` 在全局区；`x`、`p`、`c` 在栈区；`*p` 在堆区；仅 `*p` 需手动 `delete` 22. `int temp=*a; *a=*b; *b=temp;`；两种调用后均得 `a=20,b=10` 23.（1）`f1`、`f3` 安全；（2）输出 `30` 24. 依次为 `g1()`、`g1(int)`、`g1(double)`、`g1(int,double)`、`g1(double,int)` 25. `new int[5]`，`p[i]=i*i`，和为 `30`，`delete[] p`；`p` 在栈，数组在堆
提升单选	26. D　27. B　28. D
提升多选	29. AC　30. ABD
提升填空	31. `ff 10 10____7|7`　32. `016`，2 块　33. `4 1`
提升综合	34.（1）`100 6 102`；（2）`total`、`n` 全局区，`r` 是 `n` 的别名（`r` 本身不占独立对象），`p` 栈，`p[2]` 堆；（3）返回悬空引用 35.（1）`Tom_____85.3 / Amy_____92.0`；（2）数组长度需编译期常量，`const` 常量不可改；（3）读入错位，改用 `getline`
跨学科	36. B　37. ABD　38. `230.4`

解析 A 基础巩固（第 1–25 题）

第 1 题 基础 · 单选 答案：B

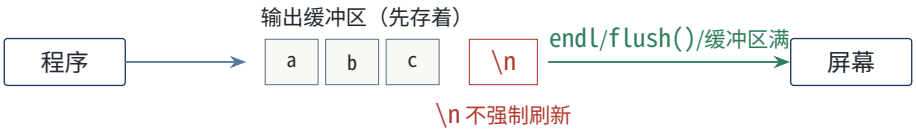
知识点 cin>> 按“类型”从输入流取数据，空白（空格、回车、制表）作为分隔被跳过。
思路 四个变量依次读：char 只取 1 个字符，int 取一段数字。读 c1 得 '1'，跳过空格读 c2 得 '2'，再跳过空格，int a 吃掉 34（遇到空格停），float b 得 56.78。
选项 A 2：那是 c2 的字符。✘ C 12：把 c1c2 当成一个整数，char 做不到。✘ D 234：混淆了字符与数字的边界。✘ B 34：✔



易错 把“按字符”和“按数字”读混；char 永远只读一个字符。　**一句话** char 读一个，int 读一段，空白自动跳过。

第 2 题 基础 · 单选 答案：B

知识点 输出缓冲区与刷新。
选项 A ✘ '\n' 只换行，endl 换行并刷新，不等价。　B ✔ 缓冲区满、endl、flush()、程序结束等才一起输出。　C ✘ 逗号表达式，cout<<a 之后 b,c 只是求值，没有输出。　D ✘ cout 有缓冲区。



易错 认为 endl 与 '\n' 完全一样。　**一句话** cout 先存后出，endl = 换行 + 刷新。

第 3 题 基础 · 单选 答案：B

知识点 setw(n) 只对紧随其后的一个输出项生效，默认右对齐，宽度不足 n 时用空格补在左边。
思路 setw(6)<<18：18 占 2 格，左补 4 个空格；'|' 与后面的 18 不再受 setw 影响。

A. 1 8 | 1 8　✘ 忽略了 setw(6)

B. ␣ ␣ ␣ ␣ 1 8 | 1 8　✔ 前 6 格含 4 空格 + 18

C. ␣ ␣ ␣ ␣ 1 8 | ␣ ␣ ␣ ␣ 1 8　✘ 以为第二个 18 也宽 6

D. 1 8 ␣ ␣ ␣ ␣ | 1 8　✘ 把右对齐当成左对齐

易错 以为 setw 对后面所有输出都有效（只管下一个）。　**一句话** setw 一次性，默认右对齐。

第 4 题 基础 · 单选 答案：B

知识点 const 变量必须初始化，初始化后不能再赋值；初值可以是运行时才算出的表达式。
思路 B 中 2+b 虽然含变量 b，但这只是给 a 一个初值，之后 a 不再改变，合法。

A `const int a;` ✘　　B `const int a = 2 + b;` ✔

C `const int a = 3; a = 4;` ✘　　D `const int a; a = 3;` ✘

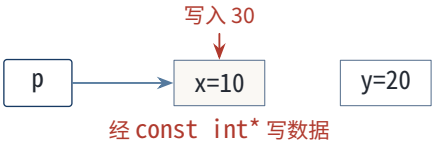
选项 A、D：没有初始化。　C：对 const 赋值。

易错 以为 const 的初值必须是编译期常量（数组长度才需要，普通 const 不需要）。　**一句话** const：定义时给值，之后只读。

第 5 题 基础 · 单选 答案：B

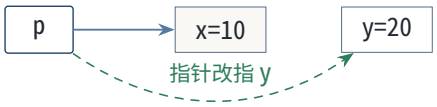
知识点 const int *p：常量指针——指向的数据不可改，指针本身可改。
思路 读法：const 修饰 int，所以 *p 只读；p 自己未被 const 修饰，可改指向。

A.



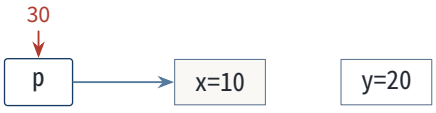
✘ 不能经 p 改数据

B.



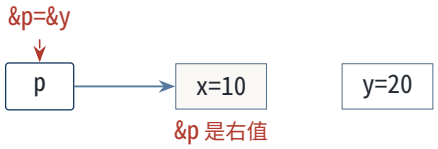
✔ 改指向可以

C.



✘ 整数不能赋给指针

D.



✘ &p 是地址值，不能被赋值

易错 把常量指针（const int*）与指针常量（int *const）弄反；口诀：const 在 * 左边锁数据，在右边锁指针。　**一句话** 看 const 在 * 的哪一侧。

第 6 题 基础 · 单选 答案：B

知识点 static 局部变量生命期延长到整个程序，存放在**全局区**（静态区）；作用域仍只在函数内。

A.



✘ 代码区存机器指令

B.



✔ static 变量在全局区

C.



✘ 普通局部变量才在栈

D.



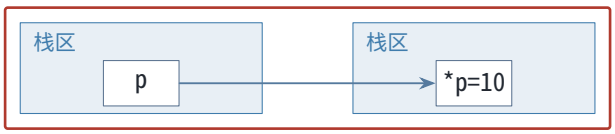
✘ 堆只放 new 出来的数据

易错 因为写在函数里就以为在栈上。　**一句话** static 改变了“住哪”，没有改变“谁能看见”。

第 7 题 基础 · 单选 答案：B

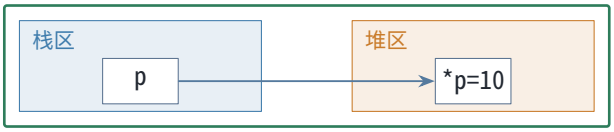
知识点 new 在堆上开辟数据，返回地址；保存地址的指针变量本身是普通局部变量，在栈上。

A.



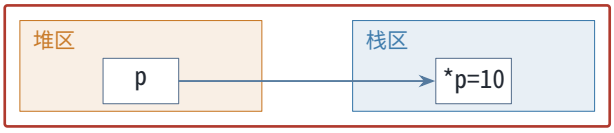
✘ 数据不在栈

B.



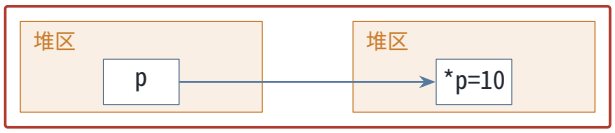
✔ 指针在栈，数据在堆

C.



✘ 正好反了

D.



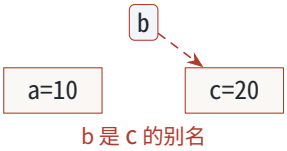
✘ 指针本身不在堆

易错 说“指针指向堆，所以指针在堆”。　**一句话** 指针变量在栈，它指向的 new 数据在堆。

第 8 题 基础 · 单选 答案：B

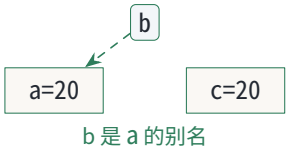
知识点 引用一旦绑定就不能改绑；b=c 是**赋值**，不是“让 b 改成 c 的别名”。
思路 b 是 a 的别名，b=c 等价于 a=c，所以 a 变成 20，c 仍是 20，b 仍依附 a。

A.



✘ 引用不能改绑到 c

B.



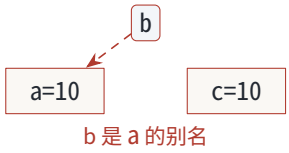
✔ a 被赋为 20

C.

编译错误 ×

✘ 语法没有问题

D.



✘ c 不会变成 10

易错 以为 b=c 能让 b 改为 c 的别名。　**一句话** 引用终身依附一个变量，后面的 = 都是给它赋值。

第 9 题 基础 · 单选 答案：C

知识点 重载看**参数表**（个数、类型、顺序），不看返回类型和形参名。

A. `int f(int)` ⟷ `void f(int)` ✘ 只有返回类型不同

B. `f(int a)` ⟷ `f(int b)` ✘ 只有形参名不同

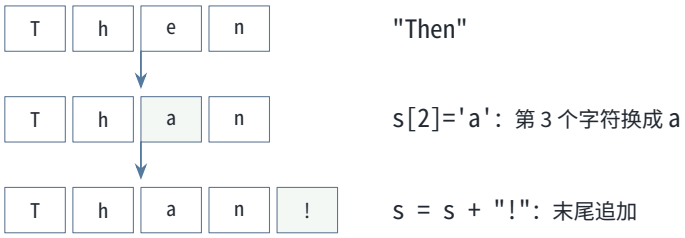
C. `f(int,double)` ⟷ `f(double,int)` ✔ 参数类型顺序不同

D. `void f(int)` ⟷ `int f(int)` ✘ 与 A 同理

易错 以为返回值不同就算重载。　**一句话** 重载 = 同名 + 参数表不同。

第 10 题　基础 · 单选　答案：B

知识点　string 可用下标修改字符，也可用 + 拼接。



选项　A 漏了 s[2]='a'；C 漏了追加；D 无语法错误。**易错**　下标从 0 开始，s[2] 是第 3 个字符。　**一句话**　下标 0 起，string 可改可拼。

第 11 题　基础 · 多选　答案：ACD

知识点　cin 的链式读取与空白跳过；endl 与缓冲区。

A cin>>a>>b；可跨行输入 ✔　　B cin>>a ✘

b; 读入 a 和 b b；读入 a 和 b ✘　　C 读 char 时跳过空白 ✔

D endl = 换行 + 刷新 ✔

选项　B 是逗号表达式：(cin>>a)，b，b 根本没被读，编译器还会给出警告。
易错　把逗号当成 >> 的连续读取。　**一句话**　连读要用 >>，不要用逗号。

第 12 题　基础 · 多选　答案：AB

知识点　全局变量与 const 全局常量都在全局区；局部 const 在栈；new 的数据在堆。



✔ g：全局变量



✔ G：全局 const



✘ c：局部 const 在栈



✘ *p：在堆，不在全局区

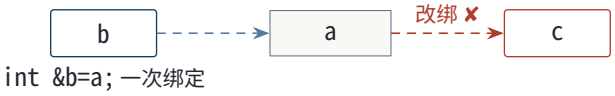
易错　以为 const 局部变量也进“常量区”。　**一句话**　位置由“在哪定义、怎么创建”决定，const 只决定可不可改。

第 13 题　基础 · 多选　答案：ACD

知识点　引用的四条规则：必须初始化；绑定后不能改绑；可作函数返回值；本质是指针常量 int *const。

A 必须在定义时初始化 ✔　　B 可以改为别的变量的别名 ✘

C 可作函数返回值 ✔　　D 本质是 int *const ✔



易错 B：b=c 是赋值而非改绑（见第 8 题）。　**一句话** 引用 = 一辈子只认一个主人的别名。

第 14 题　基础 · 多选　答案：AC

知识点　默认参数必须从右往左连续：一旦某个形参有默认值，它右边所有形参都必须有默认值。`void f(int a, int = 3)` 里不写形参名也合法。

A. 必填 必填
✔ 必填 + 默认，靠右

B. 必填 必填
✘ 默认值在左，右边却必填

C. 必填 必填
✔ 无名形参也可带默认值

D. 必填 必填 必填
✘ 必填的 c 在默认之后

易错　以为“默认参数可以任意位置”。　**一句话**　默认值只能出现在“右尾”。

第 15 题　基础 · 多选　答案：ACD

知识点　返回值是否安全，看它指向/引用的东西在函数结束后是否还活着。

A. a (f) —拷贝→ x (main)
✔ 返回值被拷贝出来

B. p (main) → a 已消亡
✘ 返回局部变量地址，悬空

C. r (main) → static a
✔ static 在全局区，函数结束仍在

D. p (main) → 堆 10
✔ 堆数据不随函数结束而消亡（需调用者 delete）

易错　把“返回指针”都当成危险；要看指向的是栈、静态区还是堆。　**一句话**　不要返回栈上局部变量的地址或引用。

第 16 题　基础 · 填空　答案：0.000345678

知识点　默认格式（%g 风格）最多 6 位有效数字；指数 ≥ -5 且 < 6 时用定点（小数）表示。

思路　$34.5678 \times 10^{-5} = 0.000345678$，规范化指数为 -4，在 $[-5, 6)$ 内，用小数形式；有效数字 $3, 4, 5, 6, 7, 8$ 刚好 6 位，无需舍入。



易错　以为会输出 `3.45678e-04`（指数小于 -5 时才用科学计数法）。　**一句话**　小到 10^{-5} 以下才变科学计数法。

第 17 题　基础 · 填空　答案：8

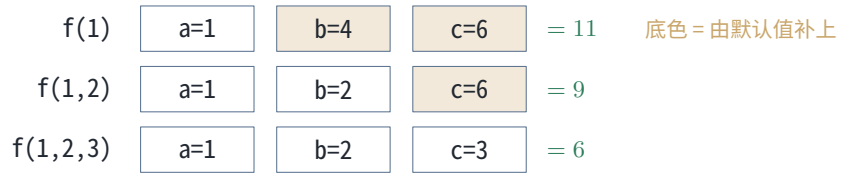
知识点　常量引用可绑定到表达式的值：编译器创建一个匿名临时变量保存 i+3 的结果，a 绑定它，与 i 再无关系。



易错 以为 a 是 i 的别名，输出 13。　**一句话** 绑表达式 = 绑一份临时副本。

第 18 题 基础 · 填空 答案：11、9、6

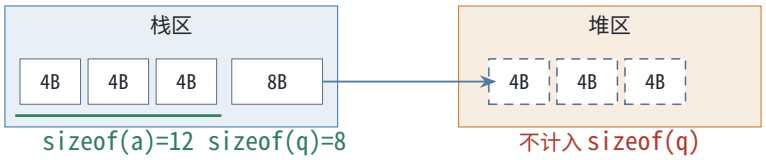
知识点 实参从左到右依次填入形参，缺少的用默认值补。



易错 以为只能省略最左边的实参。　**一句话** 实参按位置从左填，缺的从默认值补。

第 19 题 基础 · 填空 答案：12、8

知识点 sizeof 是编译期算的类型大小：数组名在 sizeof 里代表整个数组（3×4=12），指针只是一个 8 字节地址，与堆里数组多大无关。



易错 以为 sizeof(q) 是 12。　**一句话** 数组名量整个数组，指针只量指针。

第 20 题 基础 · 填空 答案：china today

知识点 string 按字典序比较（逐字符比 ASCII 码）。

思路 首字符 't'(116) > 'c'(99)，所以 s1>s2 为真，执行 swap，交换后 s1="china"，s2="today"。



易错 输出顺序漏了中间的空格或写反。　**一句话** 比较看首个不同字符，大的排后面。

第 21 题 基础 · 解答 答案：见图

知识点 四区归属；谁要手动释放：只有 new 出来的堆数据。

解答 全局区：g、G（全局 const）、s（static 局部）；栈区：x（形参）、p（指针本身）、c（局部 const）；堆区：*p。需手动 delete 的只有 *p，代码中已 delete p;。



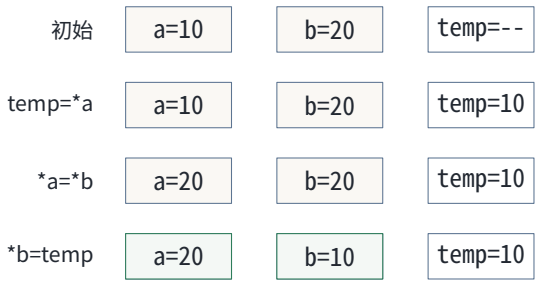
易错 p 与 *p 混为一谈；static 局部与 const 局部的位置。　**一句话** 栈自动，堆手动，全局随程序。

第 22 题 基础 · 解答　答案：见代码

错误　第 2 行 `int *temp=*a;` 把 int 赋给指针；第 4 行 `*temp` 对非指针解引用。

正确

```
void swap_02(int *a, int *b) { int temp = *a; *a = *b; *b = temp; }
void swap_03(int &a, int &b) { int temp = a;  a = b;  b = temp;  }
// swap_02(&a,&b); 或 swap_03(a,b); 后：  a=20, b=10
```

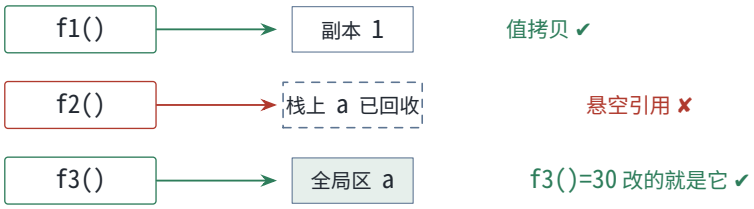


易错　指针版忘记解引用，引用版又多写了 *。　**一句话**　指针传参要 *，引用传参不要。

第 23 题 基础 · 解答　答案：见解答

（1）　f1、f3 安全：f1 返回值拷贝；f3 引用的是 static（全局区）变量。f2 返回局部变量的引用，函数结束后那块栈已回收，悬空。

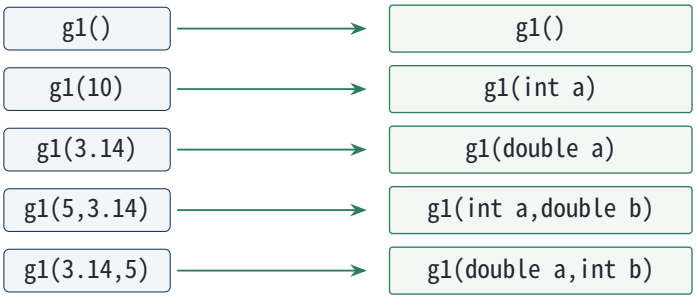
（2）　输出 30。f3() 返回 static a 的引用，函数调用是**左值**，可以放在 = 左边，等价于 a=30。



易错　以为函数调用不能出现在赋值号左边。　**一句话**　返回引用的函数调用是左值。

第 24 题 基础 · 解答　答案：见图

知识点　重载匹配：先找**精确匹配**，3.14 是 double，10 是 int。



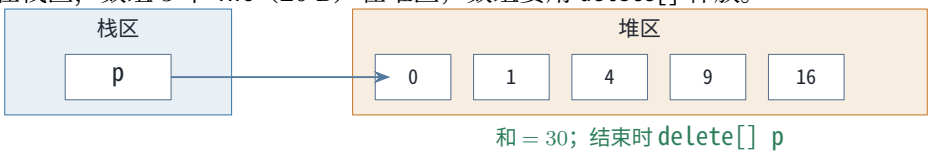
易错　3.14 误当 float；它是 double。　**一句话**　精确匹配优先，按参数类型和顺序对号入座。

第 25 题 基础 · 解答　答案：见代码

```
int *p = new int[5];
int sum = 0;
for (int i = 0; i < 5; i++) { p[i] = i * i; sum += p[i]; }
cout << sum;          // 0+1+4+9+16 = 30
```

`delete[] p;`

位置 指针 p 在栈区，数组 5 个 int（20 B）在堆区；数组要用 delete[] 释放。



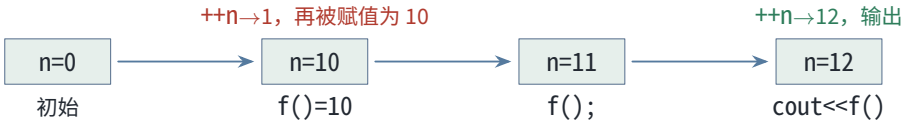
易错 用 new[] 却用 delete（少了 []）。　**一句话** new[] 配 delete[]。

解析 B 中等偏下提升（第 26–35 题）

第 26 题 提升 · 单选 答案：D

知识点 返回引用的函数调用是左值；static 变量的值在调用之间保留。

思路 f()=10：先 ++n 得 1，再把这个引用赋为 10，所以 n=10；f(); 让 n=11；cout<<f() 先 ++n 得 12。



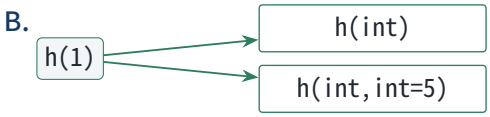
易错 以为 f()=10 之后 n 仍是 1；漏掉赋值覆盖。　**一句话** f()=10 是“先自增、再被覆盖”。

第 27 题 提升 · 单选 答案：B

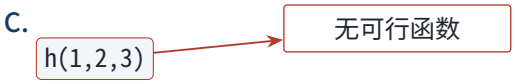
知识点 重载 + 默认参数相互作用：若一次调用能同时匹配多个候选且无最优者，则报二义性。



只有第二个可行，合法，无二义性



✔ 两个都可行且同样好，二义性



参数过多：no matching function



缺参数：no matching function

易错 看到“编译错误”就选，忽略题目问的是哪一种错误。　**一句话** 二义性 = 多个候选一样好，编译器无法挑。

第 28 题 提升 · 单选 答案：D

知识点 非 const 左值引用只能绑定左值变量；const 引用可绑定右值（临时量）。



✔ 常量引用绑字面量



✔ 常量引用绑表达式



✔ 左值绑定



✘ a+1 是右值，不能绑给 int&

易错 以为所有引用都能绑表达式。　**一句话** 想接受临时值，就写 const T&。

第 29 题　提升 · 多选　答案：AC

知识点　delete 释放的是堆块，不会改动任何指针的值；所有指向该块的指针都悬空。

A q 成为悬空指针 ✔　　B 此时 *q 读取是安全的 ✘

C delete 后置 nullptr（p 与 q）✔　　D 发生了内存泄漏 ✘



易错　以为 delete p 后 q 仍可用；或误判成泄漏（块已释放，不是泄漏）。　**一句话**　delete 一次，所有别名都作废。

第 30 题　提升 · 多选　答案：ABD

知识点　引用的各种形态：指针的引用、数组的引用、常量引用。**引用数组** 不存在，但**数组的引用**存在。

A int *&pt = p; ✔　　B int (&b)[5] = a; ✔

C char &rc[6] = c; ✘　　D const int &r = i + 3; ✔

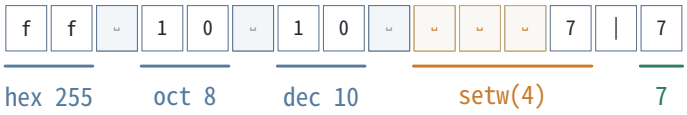
选项　A：pt 是指针 p 的别名。B：括号使 b 先成为引用，再指向 5 元素数组。C：&rc[6] 解析为“6 个引用的数组”，非法；应写 char (&rc)[6]=c;。D：常量引用可绑表达式。

易错　括号位置：int (&b)[5] 与 int &b[5] 含义完全不同。　**一句话**　要引用数组，先给 & 加括号。

第 31 题　提升 · 填空　答案：ff 10 10␣␣␣7|7

知识点　hex/oct/dec 是**持续**生效的进制控制符；setw 只对下一项生效。

思路　255 的十六进制 ff；8 的八进制 10；dec 后 10 仍为 10；setw(4) 让 7 占 4 格（前 3 个空格），加上前面输出的那个空格，数字 7 前共 4 个空格；'|'；再输出 7（setw 已失效）。

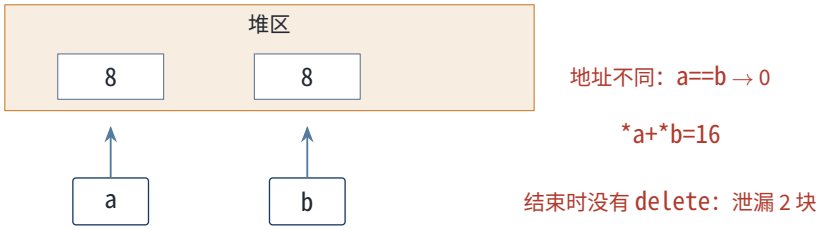


易错　以为 oct 只管一项；或忘了字符 ' ' 本身也是一个空格。　**一句话**　进制持续，宽度一次。

第 32 题　提升 · 填空　答案：016；2 块

知识点　每次 new 得到不同的堆块；指针比较比的是地址，不是内容。

思路　f() 每次开辟一块 7，加 1 变 8，返回地址。a 与 b 指向两个不同块，a==b 为 0；*a+*b=16；连在一起输出 016。两块均未 delete，泄漏 2 块。



易错　把 a==b 当成“值相等”得 1。　**一句话**　指针相等 = 地址相等。

第 33 题　提升 · 填空　答案：4　1

思路　初始 a=1；b=b+1（b 是 a 的别名）$\Rightarrow a=2$；*p=*p+2（p 指向 a）$\Rightarrow a=4$；&a==&b 为真，输出 1。



易错　把 b 当作 a 的拷贝。　**一句话**　引用、指针常量都改的是原变量。

第 34 题　提升 · 综合　答案：见下

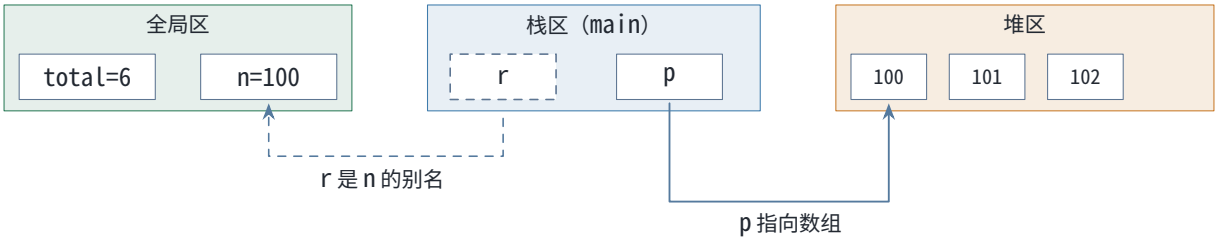
（1）　逐步跟踪 static n 与 total：

语句	n	total
bump()=10	10（先 +1 得 1，再被赋 10）	1
bump(5)	15	6
int &r=bump(0)	15（r 绑定 n）	6
r=100	100	6
fill(p,3,r)	100	6，p[0..2]=100,101,102

输出：100 6 102。

（2）　total、n 在全局区（n 是 static）；r 是 n 的别名；p 在栈；p[2] 在堆。

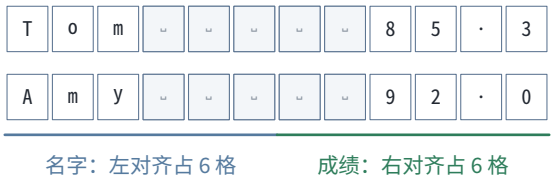
（3）　去掉 static 后 n 成为栈上局部变量，返回它的引用就是悬空引用：函数返回后 n 已消亡，r 引用非法内存，行为未定义（n 每次也都重新从 0 开始）。



易错　漏掉 bump()=10 里的“先加 1 再赋 10”。　**一句话**　static 让返回引用安全，去掉就悬空。

第 35 题　提升 · 综合　答案：见下

（1）　每行先 left<<setw(6) 输出名字（左对齐占 6 格），再 right<<setw(6) 输出成绩；fixed<<setprecision(1) 保留 1 位小数，85.26 四舍五入为 85.3，92 输出 92.0。



（2）　数组长度必须是编译期常量，const int N=2 满足；const 变量定义后不可修改，故不能在运行中改成 3，只能改源码再重新编译。

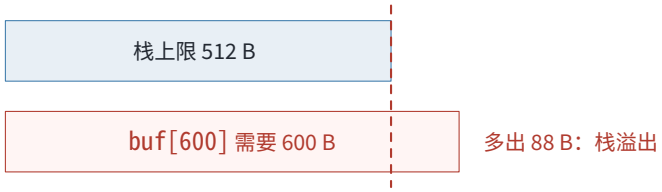
（3）　输入 Tom Lee 85.26：name[0]="Tom"；cin>>score[0] 遇到 Lee 不是数字，读入失败，score[0] 被置 0，流进入失败状态，后面的 cin>> 全部不再工作，name[1] 仍为空串、score[1] 未被写入（值不确定，实测多为 0.0）。输出类似 Tom_____0.0 与一行空名字加 0.0。

改法　整行读入再拆分，例如 getline(cin,line)，用 line.rfind(' ') 找到最后一个空格，前面是姓名，后面 stod 转成成绩。　**易错**　读入格式与程序假设不一致时，流失败后所有读入静默失效。　**一句话**　姓名含空格就不能用 cin>>，要用 getline。

解析 C 跨学科迁移（第 36–38 题）

第 36 题 跨学科 · 单选 答案：B

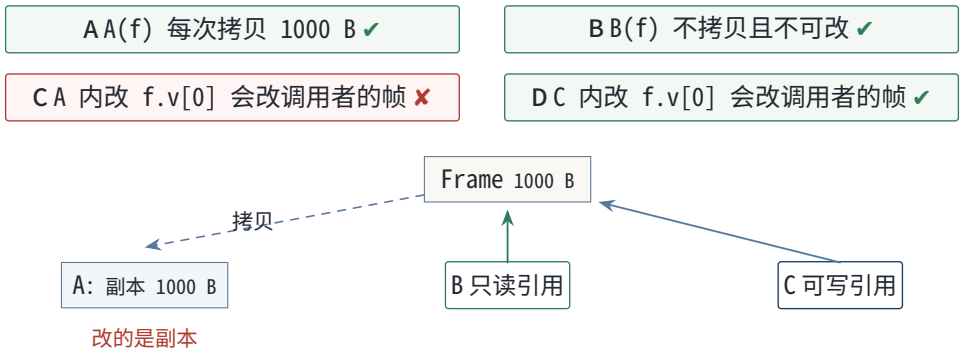
当前专题知识 栈区由编译器自动管理、容量有限；普通局部数组放在栈上；栈放不下就栈溢出。
借用的外部情境 嵌入式系统：单片机数据 RAM 很小（2048 B），要在全局区、堆、栈之间分配。
为什么不算超纲 题目已把 RAM 大小和栈上限都给出，只需用“局部数组在栈”这一条判断，不需要懂具体单片机或汇编。
迁移点 到了资源受限的系统，大数组改成 static/全局，或谨慎使用堆，是本专题“四区”知识的直接迁移。



选项 A 错：数组大小合法，编译器不知道栈的剩余空间。B 对。C 错：编译器不会擅自把局部数组搬到堆。D 错：非 static 局部数组在栈，不在全局区。
易错 以为编译器会替你选区。　**一句话** 局部数组在栈上，太大就溢出。

第 37 题 跨学科 · 多选 答案：ABD

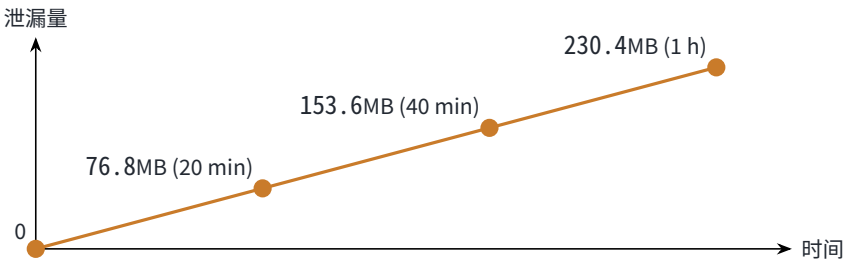
当前专题知识 值传参、引用传参、常量引用；const 防修改；拷贝成本与大小有关。
借用的外部情境 测量与信号处理：传感器每帧数据是一个固定长度的 float 数组。
为什么不算超纲 题目给出了帧大小（250 个 4 字节 float），只需做乘法并套用传参规则。
迁移点 数据量大的结构体、帧、图像，传参优先用 const T&，既省拷贝又防误改。



易错 把值传参的修改当成对原数据的修改。　**一句话** 值传参改副本，引用传参改原件，const& 只读原件。

第 38 题 跨学科 · 填空 答案：230.4

当前专题知识 new 的堆内存必须手动 delete，不释放就是内存泄漏，会持续累积。
借用的外部情境 计算机系统运维/可靠性：服务器每秒处理请求，持续泄漏会耗尽内存。
为什么不算超纲 题目给出每次泄漏 64 B、速率 1000 次/秒、时长 1 小时，只需做算术，不涉及操作系统内核。
迁移点 长时间运行的服务，哪怕每次只漏几十字节也会积少成多；养成“new 必有 delete”的习惯。
计算 $64\ \text{B} \times 1000/\text{s} \times 3600\ \text{s} = 230{,}400{,}000\ \text{B} = 230.4\ \text{MB}$。



易错 单位换算：把 MB 当成 2^{20} B（题目取 10^6）。　**一句话** 泄漏量 = 每次大小 × 次数。

第 10 章　覆盖检查表

一、知识点 × 题目覆盖表

知识点	讲义位置	覆盖题号
cin 读入（跳过空白、链式、逗号陷阱）	1.2	1、11、35
cout 与输出缓冲区、`endl`	1.3	2、11
默认输出格式（6 位有效数字、科学计数法）	1.4	16
控制符 `setw/hex/oct/fixed`	1.5	3、31、35
`const` 常变量	2.1	4
`const` 与指针（常量指针 / 指针常量）	2.2	5、33
`const` 全局 / 局部常量所在区域	2.3	12、21
四区总览	3.1	12、21、36
代码区	3.2	6
全局区（含 `static`）	3.3	6、12、21、26、34
栈区与悬空	3.4	7、15、23、34、36
堆区 `new/delete`、泄漏	3.5	7、19、25、29、32、38
引用的语法与规则	4.1–4.2	8、13、30、33
引用做函数参数	4.3	22、37
引用做函数返回值	4.4	15、23、26、34
引用的本质	4.5	13、33
常量引用与临时变量	4.6	17、28、30
默认参数	5.1	14、18、27
占位参数	5.2	14（选项 C）
函数重载与二义性	5.3	9、24、27
内联函数 `inline`	5.4	**未单独出题**（概念建议性，已在 5.4 用 `-O0/-O2` 实验演示）
`string`	6	10、20、35
综合：返回方式、内存与引用	7	34、35

二、原稿覆盖对照

原稿位置	本讲义位置
PDF 第 1–6 页（课程背景、成绩、实验目的）	前言·课程背景
PDF 第 7–18 页（最简程序、`cin`、`cout`、控制符）	第 1 章
PDF 第 19 页（`const`）；笔记 Memo 481–482、489	第 2 章
PDF 第 20–27 页（默认参数、占位参数、重载）；笔记 Memo 490–494	第 5 章
PDF 第 28–34 页（引用）；笔记 Memo 485–490	第 4 章
PDF 第 35–38 页（`string`）；Memo 482	第 6 章
PDF 第 39–40 页（`new/delete`）；笔记 Memo 481–485	第 3 章
PDF 第 41 页（致谢页）	无内容，不收录
笔记 Memo 481–494 中的原稿错误	各章“原图纠错”框（含 `iostream.h`、`void main`、逗号表达式、`String`、`func()` 缺参等）

三、可视化覆盖检查

检查项	结果
题目区配图（只画已知条件）	第 1–38 题均有
解析区配图（标注结论）	第 1–38 题均有，且与题目区分开
选项级小图（可画的选项）	第 3、5、6、7、8、9、12、14、15、27、28 题的每个选项都有小图；其中第 3、8 题在题目区也有选项图（中性配色）
其余选项为纯文字的题	以“判定条”图（☒/☒）逐选项标注：第 4、11、13、29、30、37 题
题量	38 题 = 基础 25（单选 10、多选 5、填空 5、解答 5）+ 提升 10（单选 3、多选 2、填空 3、综合 2）+ 跨学科 3（单选 1、多选 1、填空 1）